



Tower Defense with Reinforcement Learning

Reinforcement learning for intelligent pathfinding

186510

Supervised by:

Prof Andy Philippides

Dr James Knight

Department of Computer Science

University of Sussex

November, 2019

A dissertation submitted in partial fulfilment of the requirements for the degree of BSc Computer Science.



Copyright ©2020 University of Sussex

WWW.SUSSEX.AC.UK

May 26, 2020

Statement of Originality

This report is submitted as part requirement for the degree of computer science at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby give permission for a copy of this report to be loaned out to students in future years

Candidate 186510

Signed

Date May 26, 2020

To All Game Developers

You know, for saving the world.

Acknowledgements

I would like to thank both of my supervisors - Andy and James for supporting me throughout the course of writing this dissertation, especially considering this might not be the most interesting field on the world.

Besides I would also like to say thanks to my parents for their morale and financial support during the hard time.

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Motivation	1
1.3	Findings and Achievements	2
1.4	Professional Considerations	2
2	Game Development – Overview	3
2.1	Overview	3
2.2	Game Description	3
2.2.1	Typical Current Tower Defense Game Structure	3
2.2.2	Deviations	5
2.3	Requirement Analysis	5
2.3.1	Functional Requirements	5
2.3.2	Non-Functional Requirements	6
3	Game Development – Background	7
3.1	Introduction	7
3.2	Unity	7
3.3	Procedural Animation	8
3.3.1	Inverse Kinematics	9
3.3.2	Inverse Kinematics through Gradient Descent	10
4	Game Development – Software Architecture	12
4.1	Overview	12
4.1.1	Component-Based Architecture	12

4.1.2	Event System	15
4.2	Challenges – Inversion of Control	16
5	Game Development – Product Description	19
5.1	Overview	19
5.2	Basic Gameplay	19
5.3	Highlight – Procedural Animation	21
5.3.1	Leg’s Animation Details	22
5.3.2	Performance Problem	23
6	AI Development - Background Overview	24
6.1	Introduction	24
6.2	Overview	24
6.3	Pathfinding	25
6.3.1	Navmesh	26
6.4	Reinforcement Learning	26
6.4.1	Long Term Credit Assignment	27
6.4.2	Exploration and Exploitation	27
6.4.3	Advantage of Reinforcement Learning	28
6.5	Reinforcement Learning Algorithms	28
6.5.1	Value-Based Algorithms	29
6.5.2	Policy-Based Algorithms	30
6.5.3	Actor Critics	30
6.5.4	Algorithm that I Used – Proximal Policy Optimization	31
6.5.5	Curriculum Training	32
7	AI Development through Reinforcement Learning	33
7.1	Overview	33
7.2	Initial Settings	34
7.2.1	Reward Function	35
7.2.2	Output Format	35
7.2.3	Initial Input Format	35
7.2.4	Initial Curriculum	37
7.2.5	Results	37
7.3	Improvement in Input Format	38
7.3.1	Rotate Image to Encode Agent’s Orientation	40
7.3.2	Image Translation	42
7.3.3	Simplifying Color Encoding	44

7.3.4	Further Simplification – Grey Scale Color Encoding	45
7.3.5	Conclusion On Input Format	49
7.4	Improvement in Curriculum	49
7.4.1	Introduce Obstacles in the Early Levels	49
7.4.2	Randomize Obstacles in Early Level	54
7.5	Challenges	56
7.5.1	Training performance does not match actual performance	56
7.5.2	Time Consuming Nature of Reinforcement Learning	57
7.6	Conclusions	58
8	Results & Discussion & Future Work	59
8.1	Overview	59
8.2	Results	59
8.3	Future work – FABRIK	59
8.4	Future work – Improvement on Agents	60
8.4.1	Generative Adversarial Imitation Learning	60
8.4.2	Hierarchy based approach	63
	References	66
	Common Implementation on Game AI	70
	Influence Map	70
	Tactic and Planning	72
	Finite-State Machine	73
	Goal-Oriented Action Planning	73
	Simulation-Based Planning	74
	Summary	75
	Imitation Learning	76
	Behavioral Cloning	76
	Inverse Reinforcement Learning	77
	Alternatives on Software Architecture	78
	Unity Events	78
	Entity Component System	80

List of Abbreviations

ML Machine Learning	33
TD Tower Defense	1
NPC Non-Player Character.....	25
AI Artificial Intelligence.....	1
RTS Real-time Strategy	24
FSM Finite-State Machine.....	73
GOAP Goal Oriented Action Planning	73
RL Reinforcement Learning	1
ECS Entity Component System.....	12
CBA Component-based Architecture	12
OOP Object Oriented Programming.....	13
DOTS Data-oriented technology stack	81
IoC Inversion of Control	12
DI Dependency Injection.....	16
WPF Windows Presentation Framework.....	15
CM Caliburn Micro	12
GUI Graphical User Interface	78
GAIL Generative Adversarial Imitation Learning.....	25
GAN Generative adversarial network.....	60
IRL Inverse Reinforcement Learning	61
BC Behavioral Cloning.....	61
PPO Proximal Policy Optimization.....	24
A2C Advantage Actor Critic	31

TRPO Trust Region Policy Optimization	31
HRL Hierarchical Reinforcement Learning	60
IK Inverse Kinematics	7
FK Forward Kinematics	
FABRIK Forward And Backward Reaching Inverse Kinematics	10
CCD Cyclic coordinate descent	60

List of Notation

$D(x, y)$	Shortest distance between position x and y	1
$\Delta_{max}\mathcal{A}$	Maximum distance of agent's position between frames, can be think as speed of agent	35
$\mathcal{A}_i$	The position of agent in the step i	1
$\mathcal{T}_i$	The position of target in the step i	1

Introduction

1.1 | Project Overview

In this project, I have both developed a Tower Defense (TD) game and a Reinforcement Learning (RL) agent within the game. I hope by using RL, I can break the current limitation on most game Artificial Intelligence (AI) being predictable and lacking complicated tactics. I also aim to prove that it is practical to use "non-programmatic" AI to enrich players' experiences.

1.2 | Motivation

Being a keen gamer since childhood, I have witnessed improvements of video games in many different areas. From pixel arts to photo realistic rendering, from text based dialogue to stereo sound effect from the environment, game developers has made incredible improvement in various areas – improvements in AI is also significant, however it is community consensus that game AI is still generally speaking, dumb and predictable.

On the other hand, there is also rapid improvement in recent years in AI, in particular, there are quite a few RL agents that consistently beat human players which were thought to be impossible. This makes me wonder why is it the case that not many games are utilizing AI and gaining commercial success. Hence I want to develop my own game to see the intricacies of incorporating RL into game development. This means that I wanted to focus on building a working game prototype, that uses RL in the game AI.

The game is a TD game, where enemies' target is to reach the target position and the player's goal is to defend the target position with turrets that can damage the enemies, this implies that our RL agent will need to perform tactical decision to approach the

target effectively. However the first stage of the **RL** agent is to approach the target while circumventing various obstacles.

This project will give me the opportunities to delve into various aspect of game development and be one of the pioneer of utilizing **RL** in game development. Due to the two part nature of the project, this thesis will be split into two parts after the introduction. I will first discuss the development of the **TD** game itself, I will then discuss about how we developed our **AI** through **RL** based on the discussed requirement of the game.

1.3 | Findings and Achievements

In this project I used **RL** to generate an **AI** opponent in a **TD** game I built from scratch. While the performance of the trained agent is rather underwhelming, this is at least in part due to the lack of computational power which is typically needed to generate **AI** agents via **RL** in a realistic game. However we were able to investigate several factors which affect the agents that are produced by **RL**. In particular, we looked at the influence of the encoding of the input data and the curriculum(the layout of the training environment depending on the agents performance during training) used for **RL**. We saw that both aspects have important impacts on the performance and robustness on training an **RL** agent.

1.4 | Professional Considerations

The research area is one that the author can claim some competence in having studied the Fundamental of Machine Learning and Programming Analysis module taught at the University of Sussex. (BCS Code of Conduct 2.a., 2.b)

This research does not involve the use of data that could potentially be of sensitive nature. (BCS Code of Conduct 3.a,)

All claims of relevant parties will be well documented and referenced as to avoid injury to others or their reputation. (BCS Code of Conduct 2.b, BCS Code of Conduct 2.f)

Game Development – Overview

2.1 | Overview

Our main purpose of the project is to incorporate **RL** into a game, which means we have to both develop a game and the **RL** agent. Given the limited timeframe, the game has to be both simple to implement yet the AI will still need to have a basic level of complexity such that it is challenging enough for the AI to learn – otherwise we could not prove that it is indeed practical to incorporate **RL** in game development.

Base on this requirement, I have decided to create a 3D **TD** game with some tweaks on some of the existing conventions. In the following sections, we will first outline what our game will be like, followed by a requirement analysis, and then finally we will briefly discuss what technology is used while creating the game.

2.2 | Game Description

As mentioned in section 2.1, I will be building a **TD** game with a few tweaks as our platform for training our **RL** agent. In this section I will first briefly introduce what is a **TD** game, and then I will elaborate how a few fundamental rule is changed for our purpose.

2.2.1 | Typical Current Tower Defense Game Structure

TD has been a long-living subgenre of video games, which emerged in the early stages of video games history and is still popular today. A typical **TD** game often looks like the screenshot shown in figure 2.1 and is generally structured as follows(graphical representation can be seen in figure 2.2):



Figure 2.1: Screenshot from *Flash Element*, a classic representation of tower defense game

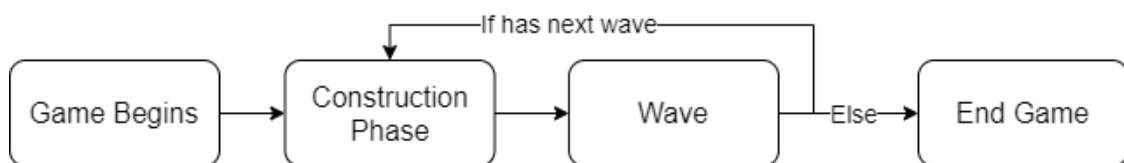


Figure 2.2: Typical game flow for TD game

1. Game begins, the player is given a certain amount of money
2. The *Construction Phase* - Player will construct turrets at different locations on the map, various types of turret are available through upgrading from the “base turret”, they all serve different purposes such as dealing damage or slowing down the enemy. Depend on the specific implementations, the player has unlimited or limited time to think how they should organize their defends for the next “wave”
3. The *Wave* begins - Enemies are spawned and start marching with a predefined path to the player’s base. Every time an enemy made contact with the player’s base, a certain amount of health of the player is reduced. The wave always consists of a fixed number of enemies and the number increases as the game progress. Game ends when a player loses all his health or when the last wave is finished (every enemy of the last wave is killed), the player can still upgrade turret on the fly when they earn enough resources through killing the enemies.
4. Go back to construction phase if there is another wave, else the game/level is finished.

2.2.2 | Deviations

From the previous subsection, one can immediately see that there is not much room for **RL** – after all, the agent just march in a predefined path. To allow our **RL** agent to shine, I have made a deviation such that the enemies need to make contact with the constantly moving player character itself, instead of a static base that cannot move. Besides, the battlefield will be significantly more open with less obstacles, which means there are more paths that can reach the player, with some paths potentially being significantly less risky (with less turrets in the path, or just short enough that the turret does not matter) than others.

The fact that numerous path is available means that the **AI** needs to be *tactical* to be effective. For example, taking the shortest path is not necessarily the best option when another longer path is much safer. The agent will need to learn the tactical aspect of the game, which is a task complicated enough to be challenging for proving the practicality of using **RL** in game development.

2.3 | Requirement Analysis

This section outline some of our requirement on our **TD** game, note this list is not comprehensive as most of the requirement of a **TD** game is too subtle to be discussed in detail. We will focus on aspects that encourages active decision making from either player or the enemies (agent) as it is what is important for our project to prove the practicality of utilizing **RL** in game development.

2.3.1 | Functional Requirements

A list of other functional requirements is as follow:

- Players should be encouraged to make active and real-time decisions during the wave.
- Enemy should be "smart" - players are discouraged to out-gun them, players need to out-smart them to survive longer, instead of simply fighting them with superior firepower.
- Different strategies should be viable to the player, such as
 - Turtling, where the player set up heavy defense in one location and very rarely get out of the fort.

- Roaming, where the player runs around the map and seek opportunities to strike at the target and doesn't rely on heavy defenses.
- Spawn Camping, where the player set up heavy defense positions on the enemy spawn point, such that any enemy spawned at that point will be killed instantly.

2.3.2 | Non-Functional Requirements

The following is a list of non-functional requirements, that can also be viewed as a "design guideline":

- In contrast to most existing TD, the target of the enemies must be mobile (controlled by player) during attack phase in a relatively open terrain.
- Turret can be placed by the player within a certain range.
- Player has some self-defending power - walking alone in the wild should be dangerous but not impossible.
- Enemy are human-like and will make decisions depending on their surroundings. Tactics such as flanking and baiting (where some objectives for the player pretends to be an easy target and springs a surprise attack when the player approaches) should be deployed by AI when necessary.
- 3D view with no global map available, allowing enemies to exploit players' limited vision.

Game Development – Background

3.1 | Introduction

As our goal is to prove that [RL](#) can be practically utilized in game development, even if our final product is merely a prototype, it needs to possess a certain level of complexity otherwise it could not function as a proof that [RL](#) can be practically used in commercial game development.

To create a slightly more polished prototype within the limited timeframe, I have used Unity throughout the entire development process as it provides various utilities that shortens the development cycle and allows me to put more focus on the [RL](#). As such we will give a brief introduction on Unity in the upcoming sections which is important to our discussion in [chapter 4](#).

Besides, to expose myself to various areas of game development. I have created and animated our enemy agent's 3D models from scratch through procedural animation. We will briefly introduce procedural animation using Inverse Kinematics ([IK](#)) through gradient descent, as it is one of the more complicated system that I built and has introduced problems that we were not able to resolve within the timeframe. The more detailed implementation and the problem we faced will be further discussed in [chapter 5](#).

3.2 | Unity

Unity is a game engine that aids game development. Being one of the most popular general purpose game engine around the world ([Toftedahl and Engström, 2019](#)), it is well-known for its ease to learn and huge community basis. Unity will be used throughout the

entire development process of both AI and the environment, this significantly reduced development time needed on rendering and physics related code.

Compared to Unreal, Unity's major competitor, Unity has significantly less development overhead and is more suitable for rapid prototyping. The fact Unity use C# also means it is a lot less error prone to C++, which is widely used in game development environment for its potential on low level optimization.

I have used many different components provided by Unity, such as lighting and particle system. Although using them itself requires skills and knowledge, we will not discuss those topics due to limit on word count.

3.3 | Procedural Animation

Contrary to the other more traditional approaches to animation such as manually key-framing, procedural animation can generate animation in real-time to create a more diverse series of actions (Green, 2016). This is more important in computer games as some animation cannot be "predefined" – for example, one cannot practically animate all possible walking animation on a highly dynamic terrain.



Figure 3.1: Different characters in Spore, all generated and animated in runtime. The fact they are created in runtime make it difficult to pre-define any animation for them.

Procedural animation is hence often used to simulate particle systems, rigid body dynamics, and other animation that tends to have a high "variety" and needs to adapt to the environment, typical examples in games includes animation for fire, or the ragdoll

physics for a fall of character. One game that yields commercial success is Spore, where all creatures created by users in runtime utilizes procedural animation.

3.3.1 | Inverse Kinematics

When working with procedural animation, there is often the need to put a certain part of the animated objects into a specific position. For example, if we want to animate a character picking up a mug, we need the character's hand in the mug's handle's position. In the process, the animated object must also adhere to certain constraints – using the same example, the joints of the character need to be in a natural angle that can be achieved by a human. This seemingly simple requirement is, in fact, a very difficult task to solve. So much so game developers resort to robotics technology – IK.

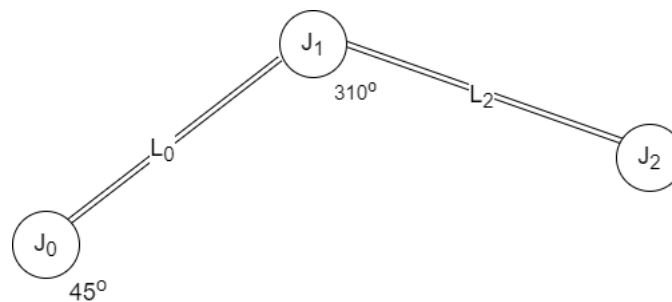


Figure 3.2: A typical linked chain, where position of the end effector J_2 is calculated by FK

In robotics, a linked chain constructed with joints $J_0 \dots J_n$ and links between joints $L_0 \dots L_{n-1}$. Given a linked chain, angles of all joints and lengths of all links, FK refers to the use of kinematics equation to find out the position of the last joint J_n (more often known as the *end effector*). Figure 3.2 shows a typical linked chain and how FK find the position of the end effector.

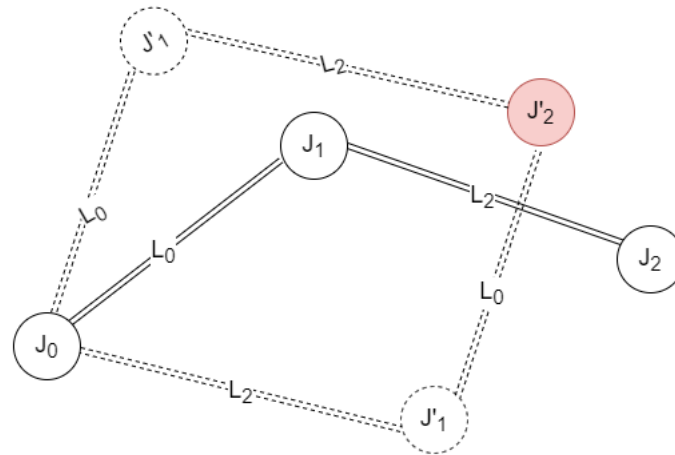


Figure 3.3: Given the same linked chain and the same target effector position J'_2 , how multiple solution can be yielded with **IK**, note although rotated, length of both L_1 and L_2 remains the same.

Intuitively, **IK** is the mathematical process that given a linked chain and the required position of the end effector, calculate the angle of all joints such that the end effector will be in the position of the required position (we refer the solution as s), this reversed operation yields the name of "Inverse" kinematics. One thing to note is that unlike **FK**, **IK** very often has multiple valid answer, and it is up to the implementation to choose the best solution. Figure 3.3 demonstrates how multiple solution can be yielded by an **IK** solver given the same input.

Typical algorithm for solving **IK** includes gradient descent and Forward And Backward Reaching Inverse Kinematics (**FABRIK**) introduced by Aristidou and Lasenby (2011). I chose to create a gradient descent **IK** solver because of its relative ease to implement and tuning, however we will discuss about how **FABRIK** can potentially improves our game in chapter 8.

3.3.2 | Inverse Kinematics through Gradient Descent

Gradient descent is an iterative optimization algorithm to find a local minimum of a function $f(x)$ (Ruder, 2016a). When used as an **IK** solver, f is a function that given an input x where x is the rotation of all joints $j_0 \dots j_{n-1}$, output the distance between j_n the end effector and the target position t .

This is a simple implementation but not without caveats, one can immediately see how the local optimum might not be the global optimum – however, this is sometimes

desired in an [IK](#) solver. This is because even if there is some other optimal solution s available that results in a solution that does not, if the configuration in s is significantly different from the current configuration, it will imply that the linked chain will need to make a huge movement to apply solution s . This huge movement is often not desired as it breaks the smooth interpolation in close vicinity we require from the [IK](#) solver, which is needed for animation purposes.

One advantage of using gradient descent is its relative ease to tune, when using an [IK](#) solver we often want to add some additional constraints, for example, it is often desired that the linked chain to use a more "natural" configuration, this naturalness is something that is harder to accomplish with other [IK](#) algorithms, but can be relatively easily achieved when using gradient descent through adding a comfort function into the function f , implying when performing gradient descent the algorithm will also take the naturalness into account([Baerlocher, 2001](#)).

The disadvantage of using gradient descent is that it can be relatively slow if one want a high quality result. We will elaborate on the performance problem in chapter [5](#).

Game Development – Software Architecture

4.1 | Overview

In the following sections I will discuss both Component-based Architecture ([CBA](#)) and Caliburn Micro ([CM](#))'s event system in detail, as [CBA](#) is used extensively during the development of our game and is the traditional approach of Unity development, and [CM](#)'s event system that we used which allows different components of the game to communicate between each other. These two elements are discussed thoroughly as it is the key to our architecture and heavily impacts not only the game development but also part of the implementation of the [RL](#) agent as well. At last we will also briefly discuss the challenge on achieving Inversion of Control ([IoC](#)), which is a programming principle that facilitates decoupling between modules.

I have also done some research over other valid alternatives, however due to limited word count I have put the discussion on those alternatives and why we chose [CBA](#) and [CM](#)'s event system over them into the appendix [8.4.2.2](#). Note implementation details such as design pattern will not be discussed as they are out of scope of discussion, instead I will focus on a more of an overview of the architecture.

4.1.1 | Component-Based Architecture

There are two architectures that is supported by Unity - the [CBA](#) which we employed, and the Entity Component System ([ECS](#)) which we shall discuss in the section [8.4.2.2](#). Unity by default is supporting both architecture, with [CBA](#) having a significantly higher adoption rate, while [ECS](#) is getting more popular as it is known that Unity will be moving

into ECS in the future.

CBA is a relatively straight-forward architecture, deviating from the traditional Object Oriented Programming (OOP) which have a deep class hierarchy, **CBA** has a significantly higher tendency of dissecting a part of an object in the world into different *components*, and we achieve the desired behaviour by having different components communicating with each other.

To put **CBA** in an example, consider the following scenario that happened during my development. In the project, I was in need to create `Unit`, which can move around the world, and will die after taking certain amount of damage. I also need to create some turrets, that cannot move around the world, but will need to shoot bullet towards the enemy agents, both turrets and units share some common methods such as rendering or collision.

When designing in **CBA**, we would have something similar to figure 4.1. An empty class known as `Entity` will hold different components, where each component is responsible for one specific tasks. In this case we will have 4 different components and different entity will simply contains a different set of components. In our case, `Unit` entity will have the `CommonComponents`, `DamagableComponent` and `MoveComponent`, and the `Turret` entity will also have a `CommonComponents` and a `ShootBulletComponent`

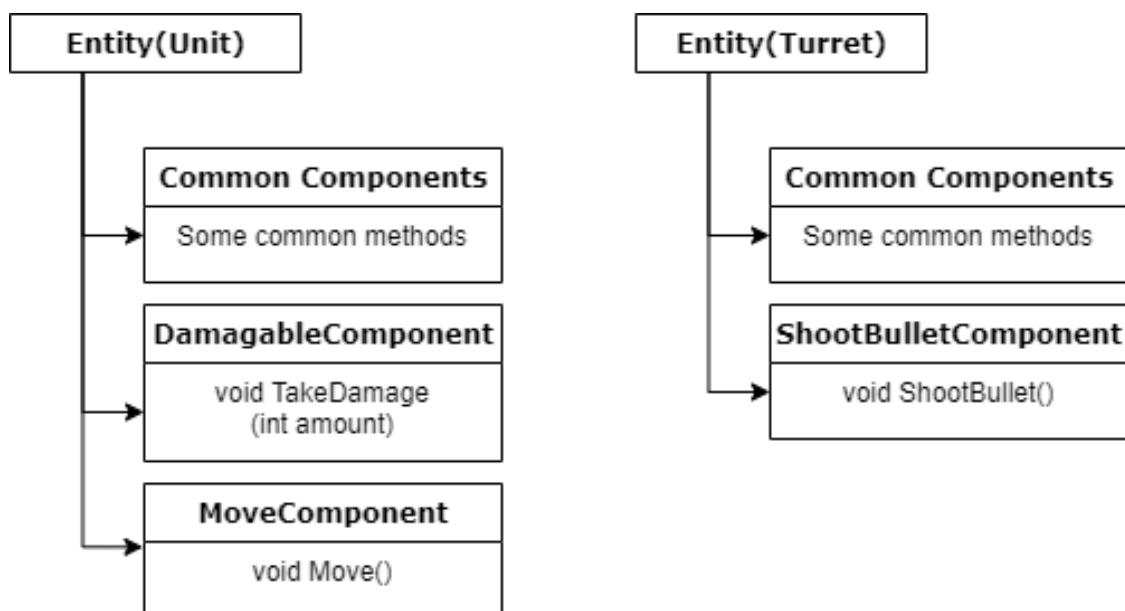


Figure 4.1: Demonstration of how to implement turrets and units in **CBA**

The reason why **CBA** is popular amongst the game developer is **CBA**'s ability to

adapt to changes which happens very often in game development world. In my case, I wanted to make a change such that my player can shoot to defend himself, thus the need for an `Unit` to shoot arises. In [CBA](#), the change would be trivial, as demonstrated in figure 4.2, all we need to do is to combine multiple existing component together, and we have our `Player` entity.

However it is much more involved in traditional [OOP](#), as we have broken our premise that *if something can move and can be damaged will not shoot bullet*. One would need to refactor the inheritance hierarchy to achieve what we have done in [CBA](#) with little effort. This is due to the nature that inheritance, although encourage re-usability, often makes the project less "modifiable" ([Sheldon et al., 2002](#)).

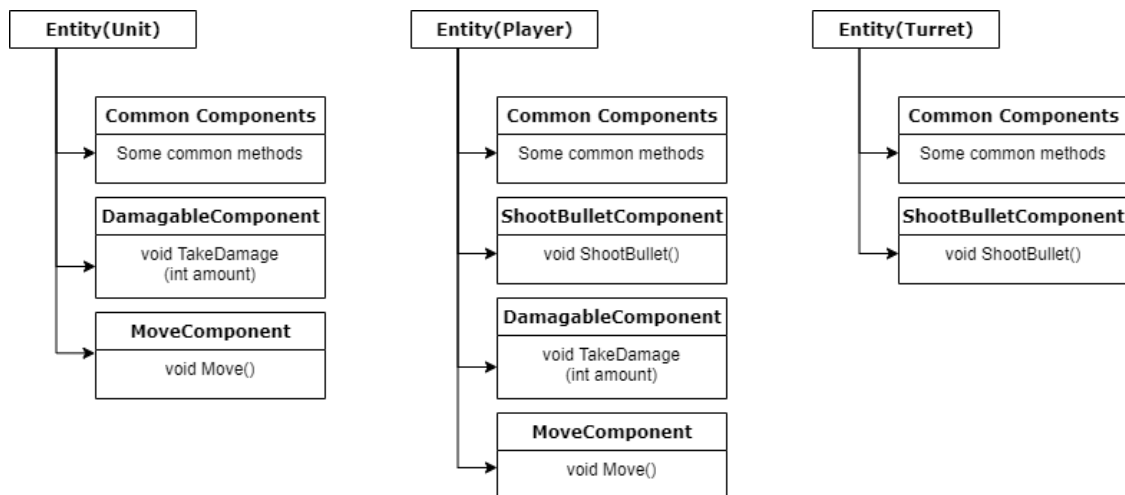


Figure 4.2: Demonstration of how [CBA](#) can adapt to the changing requirement effortlessly

One may note that even in traditional [OOP](#), many advocates *composition over inheritance* ([Erich Gamma, 1994](#)). We should notice that it is a matter of degree - in modern development, no matter which methodologies we employed there is always some mixture of other paradigms.

In general, [CBA](#) has the advantage of flexibility over traditional [OOP](#). When the need of a new type of entity arises, one can often utilize existing components to create one. However it also have the disadvantage of being in general harder to debug, as it is easy to lead to unintended behaviour when multiple component interacts with each other, and with large numbers of various components in the scene, it can be hard to find the tree within the forest.

4.1.2 | Event System

As mentioned previously, the crux of [CBA](#) is that every component is designed to work individually, ideally without other components. This implies that in most cases we should avoid them having a direct reference (known as a dependency) on other component. This is crucial to maintain the flexibility of our project.

However, there are always some cases where we want components to communicate with each other, and to prevent the need of a dependency, I have decided to use an event system.

In event-driven architecture, we describe the fact that "something happened" as an event, and entities can either "publish" an event when it happened, or "subscribe" to a type of event, so it will be notified when the respective event is published. In the process, the publisher and the subscriber of the events is completely ignorant of each others existence, and thus removed the direct dependency on each other.

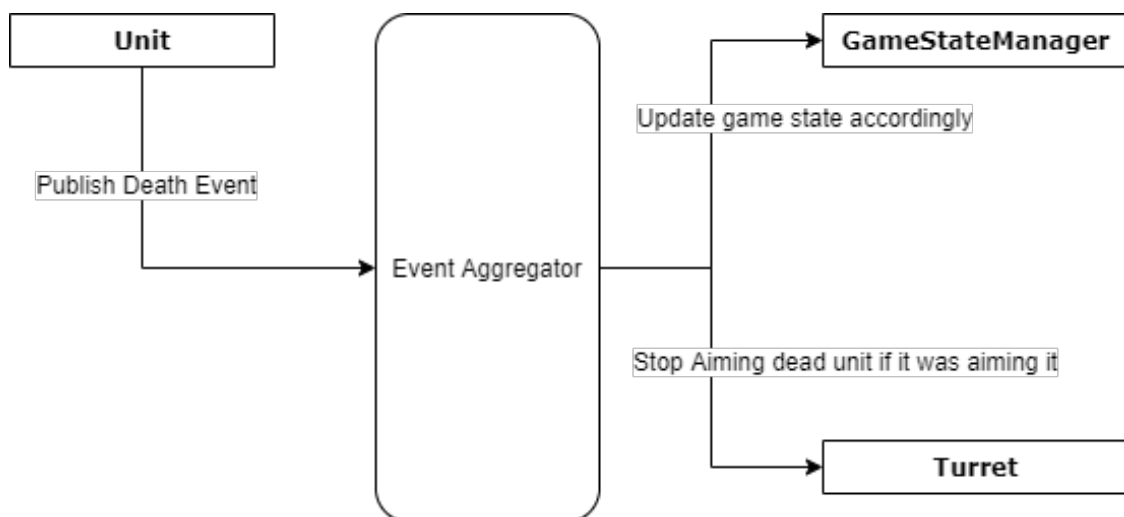


Figure 4.3: Demonstration of how event system helps to remove dependencies between Unit, GameStateManager and Turret in our project

We relies on an `EventAggregator` from [CM](#), a front-end framework in the Windows Presentation Framework ([WPF](#)) world for the concrete implementation.

We should note that this is not a silver bullet, and there will still be cases where we need a reference to the specific object, the obvious example will be the `EventAggregator` itself - both subscriber and publisher will need a reference to `EventAggregator` to interact with the rest of the environment. We will briefly discuss how we resolved these cases in [section 4.2](#)

4.2 | Challenges – Inversion of Control

As a final note on the environment, I want to briefly mention an ongoing problem during my development when trying to achieve IoC - an important principle in software engineering.

IoC encourages an "inverted" flow of control as compared to traditional control flow in design. A real world example will be a desktop in contrast of a laptop: when you use a desktop, the desktop controls where you sit (as it is stationary), you are "controlled" by the desktop; in contrast, with a laptop, you "control" where you use it. IoC encourages in-dependency between modules, and freedom to replace modules with little to no side-effects.

In software engineering, this means that the software/object consumer gains more control over the software/object itself. Thus it also sometimes known as the "Hollywood Principle" - *Don't call us, we call you*.

There is many different ways to achieve this, one of the most popular being Dependency Injection (DI), which refers to a pattern that "any dependency of an object should be provided by the constructor or the caller". Doing so improves flexibility, maintainability and testability.

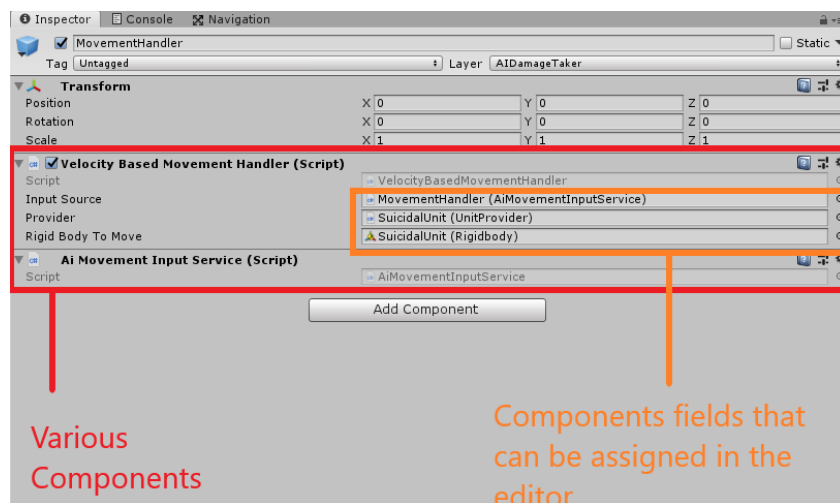


Figure 4.4: DI with Unity Inspector - step 1

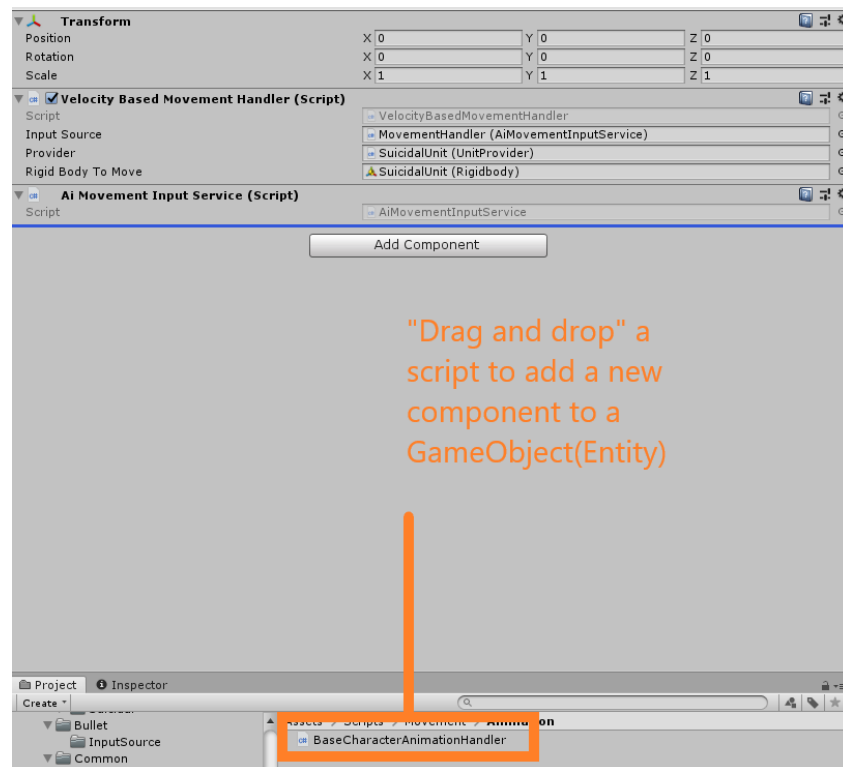


Figure 4.5: DI with Unity Inspector - step 2

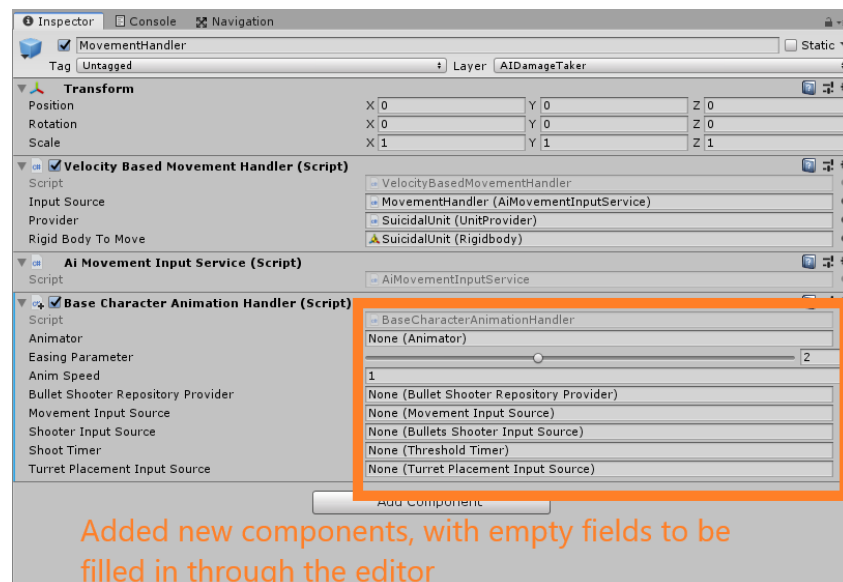


Figure 4.6: DI with Unity Inspector - step 3

In Unity, it can be achieved by using the serialization provided by Unity Engine, which as a result¹ we can achieve DI through dragging and dropping components in the Unity inspector, as demonstrated in figure 4.4, 4.5 and 4.6.

However, DI is inherently difficult to be *effectively* implemented in Unity. While one can achieve DI using the Unity Inspector, there are few huge limitation with Unity:

1. The injected dependency must not be an interface. Implies that the class with ultimately relies on the implementation of another class
2. The injected dependency must be serializable by Unity, which means it does not support generics, and must either inherit from `MonoBehaviour` or `ScriptableObject`², or is a primitive type.

One of the industry standard to solve this problem is to use Zenject³, a DI framework for Unity, which allows developer to implement DI without the aforementioned restrictions. It also provides other features such as event system and memory pools.

I decided to not use it, despite my previous experience with Zenject. Zenject is a very powerful framework indeed but do come with a significant overhead, and for prototyping purpose of this project, I assumed that Zenject is an overkill - although I agree Zenject will definitely proves to be helpful once the project scales.

Instead I tried to achieve DI with extended usage of `ScriptableObject` as suggested by both Fine (2016) and Hipple (2017). The result is *satisfactory*, but at the end of the project asset handling and boilerplate do slowly get out of controls. Besides, it still suffer from the restriction on the reliance on Unity's serialization.

¹Details of how it is handled is out of scope of discussion, we will refrain ourselves to discuss only on the practical implementation

²Can in principle inherit from `UnityEngine.Object`, but is an edge cases and is very difficult in practice. It is also mentioned in the [official documents](#) that it is not intended to be widely used in scripts

³Pokemon Go, a famous mobile game, used Zenject for its development and has publicly promote the usage of Zenject in [Unite 2016](#)

Game Development – Product Description

5.1 | Overview

In this chapter, I will first elaborate on how the final prototype looks like, and then I will briefly discuss the procedural animation which are the slightly more advanced techniques that I implemented from scratch. After this chapter, we will progress into discussion on the development of RL agent, which heavily depends on how the game itself is set up.

5.2 | Basic Gameplay

Our basic game flow largely remains the same with what I have described in figure 2.2. In my implementation, the player is free to roam around the map, however is subject to attack from the enemy agent. Whenever it is in contact with an enemy player will be damaged and loses some health, when the player loses all his/her health the game is over. The player's goal is to survive as long as it can, and is given a score depends on how long it survived. Figure 5.1 is a screenshot that shows how the game looks like.

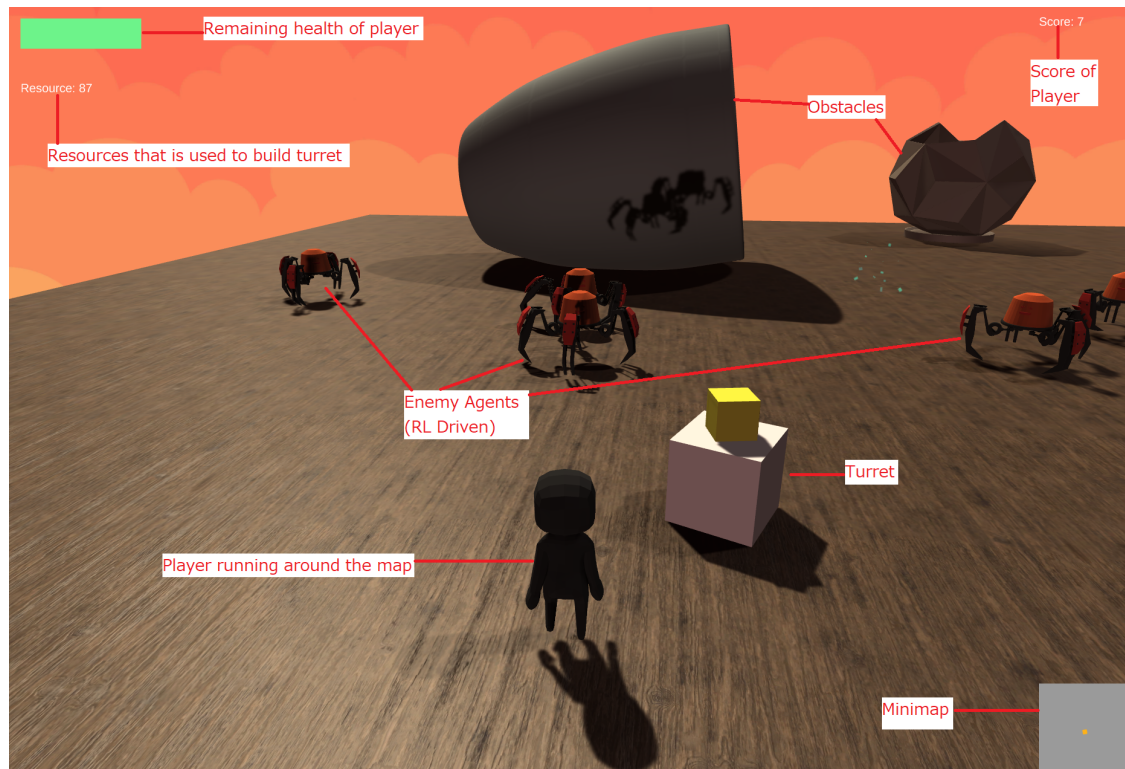


Figure 5.1: A screenshot of our prototype, demonstrating how the basic UI works and how the game looks like.

For the player to defend itself, the player can either shoot bullets that can damage the enemy robots, or to use resources that will accumulate over time to build turrets that can shoot at enemies within range. Turrets can only be placed right in front of the player, as such the player will need to run to the desired position for the turrets in order for him to build it. The turrets can also upgrade to one of the three variants, to either increase the range, deal more damage or to shoot faster, yet again to upgrade the turrets, the player needs to stand next to the turret. Figure 5.2 shows how the player can upgrade the turrets.

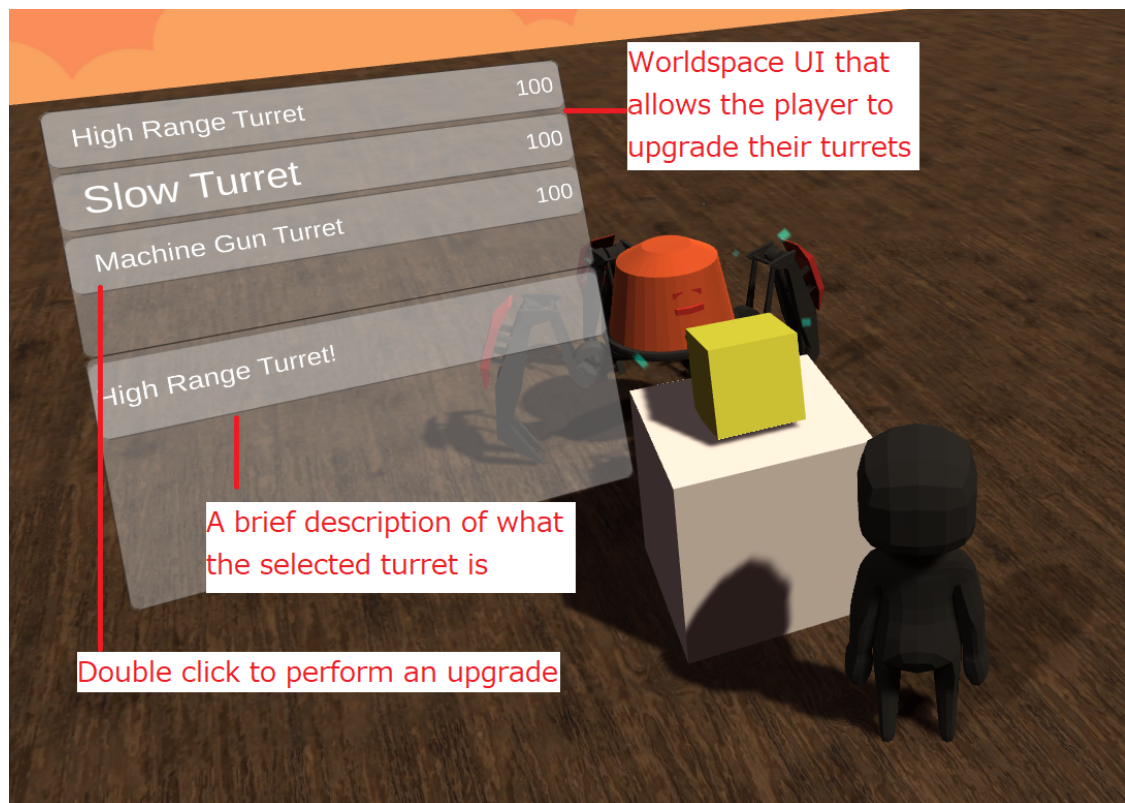


Figure 5.2: A screenshot of showing how turret upgrade can be performed

5.3 | Highlight – Procedural Animation

The enemy agent shown in figure 5.1 is animated through procedural animation. Whilst the detailed development process is out of the scope of discussion due to word limit, the animation specifically on the legs has introduced some problems during our development, and hence worth some discussion.

Before proceeding, I recommend looking at the animation in action in the following [youtube link](#), I will discuss how the leg is animated through **IK** in the following sections, and what problem **IK** has introduced. We will also discuss how we can potentially solve this problem in the future work section.

5.3.1 | Leg's Animation Details

Instead of directly animating the leg itself, I animated the legs' target end effector's position(e'), such that the leg is under a constraint that it must align itself in a way that the end effector position e is as close to e' as possible. To achieve this result, we have used **IK** to find the required rotation of each joint of the leg such that the constraint can be satisfied. Figure 5.3 shows how the leg of the robots is aligned such that it stands on the target position of the end effector.

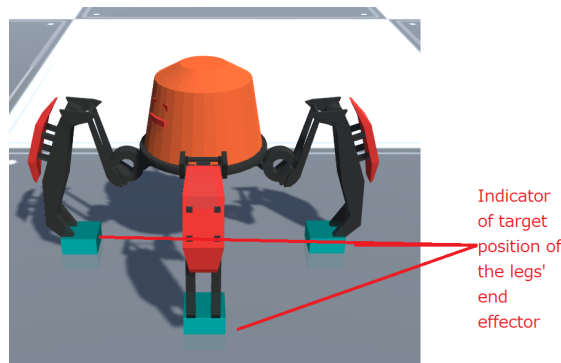


Figure 5.3: Demonstration of the target position and the robot's leg's alignment, note how each of the end of the leg stands exactly on top of the blue gizmos indicating the target position for the end effector of the legs.

In our implementation, e' of all four robot legs remains the same until the distance between e' and the center of the robot c is greater than certain threshold t . Once the distance between e' and c exceeds the threshold t . We calculate a new target end effector's position e'' such that distance between e'' and c is at a comfortable value v (which is a hyper parameter we can customize). After we calculated e'' , we perform a quadratic bezier easing between e' , e'' and a middle point m between e' and e'' , m is lifted from the ground by a custom value h such that the leg will be lifted above the ground during the animation. This process is demonstrated in figure 5.4. Through animating all 4 legs in the same manner and other subtle animation which we did not covered due to word limit, we get the illusion that the robot is walking around, when in fact we are only animating the legs end effectors.

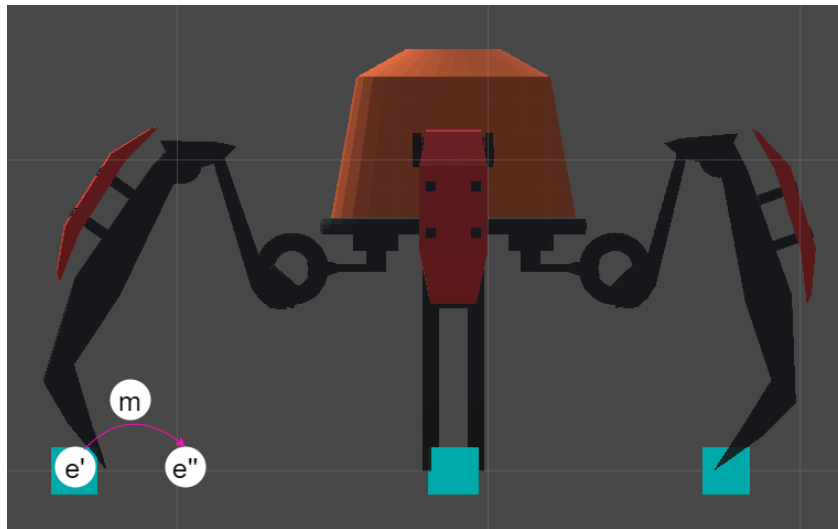


Figure 5.4: Demonstration of how the leg's alignment will be interpolated between e' , e'' and m .

5.3.2 | Performance Problem

As mentioned in chapter 3, one caveat of using gradient descent as an IK solver is that it can potentially be quite slow. This has become a problem in our game where when too many robots exist in the same scene, the IK solver becomes the performance bottleneck and leads to a visible drop in frame rate. This is obviously a problem as even using my machine which is reasonably potent (CPU: i5-9600k), the frame rate drop is too apparent that it introduces unpleasant experience during gameplay, however given the limited time frame we did not have the time to rectify this problem, yet I will discuss how it can potentially be solved in chapter 8.

AI Development - Background Overview

6.1 | Introduction

In previous chapters, we have discussed how the game is implemented and how the TD game works. Starting from this chapter we will put our focus on the development of using RL for our enemy agent.

In this project, I aim at making an opponent AI possess "human-like" behaviour - to put it simply, we are satisfied as long as the player finds the opponent non-robotic and is challenging enough to play against.

To create a human like AI agent, I have decided to use RL to generate our enemy AI. Considering that we are using Unity to create the game, it is natural to use Unity ML Agent to handle to RL training procedure, as Unity ML Agent has implemented Proximal Policy Optimization (PPO), the algorithm that we used for our training section and has nice compatibility with Unity Engine itself.

6.2 | Overview

In the following sections, we will given a brief introduction on the navmesh and general RL algorithms. Pathfinding is discussed as we uses navmesh to construct our reward function(which will be discussed in chapter 7) and its fundamental importance in game AI in general, while general knowledge on RL is vital to understand the training process which is discussed in chapter 7.

Because of word count, we also summarised the common approaches to implement Real-time Strategy (RTS)'s AI in appendixes. RTS's AI is discussed to provide a highlight

on some of the caveats on the existing approaches. RTS's AI is chosen because RTS game in general, require highly intelligent behaviour for many individual units in the game, which is a similar situation to our requirement.

Details of imitation learning can also be seen in the appendix 8.4.2.2, they are included as although we used Generative Adversarial Imitation Learning (GAIL), a type of imitation learning in our development during our development, the results is not satisfactory enough to warrant further investigation given the limited timeframe. However GAIL itself will be discussed thoroughly in chapter 8.

6.3 | Pathfinding

Pathfinding is one of the more thoroughly researched topics in game development. This is due to the fact that many games will benefit from having an Non-Player Characters (NPCs) running around the map in a seemingly *intelligent* manner.

It is worth noting that intelligent pathfinding does not necessarily imply using the shortest path to navigate to the target - in fact in most cases, it is not! NPCs often needs to take a longer route to either avoid danger, pretend to be exploring the world, or even just simply mimicking human (as human often does not take the shortest path).

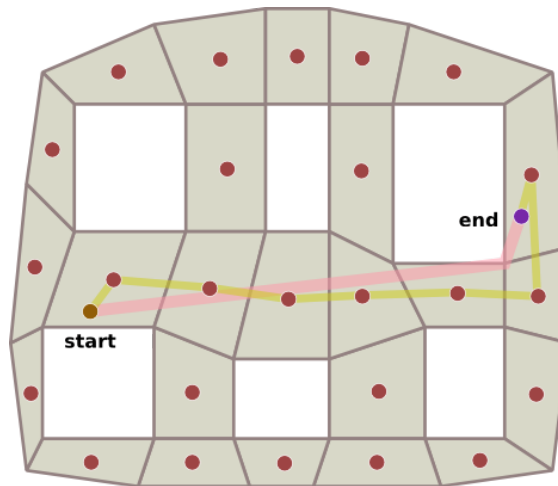


Figure 6.1: Graphical representation of navmesh created by Patel (2009), yellow and red lines indicate path before and after string-pulling optimization respectively

In the most simplified manner, most of the current practice involves creating a mesh of convex polygons where agents can move freely within, where regions that are next to each other are connected with an edge - it is known as a navmesh. Figure 6.1 demonstrates

a typical navmesh and its usage. More about the topic could be found in article written by [Patel \(2009\)](#).

After creating the navmesh at design time, the game's pathfinding algorithm will be running on that mesh in run time, finding a suitable path whenever necessary. A* pathfinding is often used as it has good performance and easy to implement, its implementation is beyond the scope of our discussion, if more information is needed please refer to the article written by [Cui and Shi \(2011\)](#).

6.3.1 | Navmesh

As aforementioned, navmesh needs to be created at design time, there are various approach on generating navmesh ([Rabin, 2014](#)) and each aims at optimizing on a slightly different problems, for example, [Sturtevant \(2014\)](#) mentioned an approach they implemented for *Dragon Age* aiming at optimizing memory usage. Another alternative is movement-based approach that is relatively easy to implement and works well in various conditions - but requires hours to build ([Axelrod, 2014](#)).

However, it is fair to say for any variant of navmesh, they will all inevitably suffer from the following problems:

- When there is an update to the games geometry or collision model, such as an impassable object is being destroyed, or a bridge is built between two originally disconnected areas, the navmesh will need to be updated in order for the pathfinding algorithms to function properly
- Optimization requires a lot of effort, many resources need to be invested into it to create a navmesh that suits the need of a particular game
- By itself, it does not tackle the problem of having *intelligent* pathfinding, without more information it can only serve as a tool to find the shortest possible path. (Note other techniques that cover this problem is discussed in appendix 8.4.2.2)

To conclude, although navmesh is a powerful tool for pathfinding, it comes with a great cost and can be difficult to implement - in some cases tuning will be required and could take a huge amount of resources from the development team ([Hamm, 2014](#)).

6.4 | Reinforcement Learning

Machine learning algorithms aims to build a mathematical model that performs a certain task based on sample data (more often known as training data). RL is one of the machine

learning paradigm that aims to train software agent to take actions in an environment so as to maximize some cumulative "rewards". Machine learning related research has been thriving lately and its application can be seen in various topics, however it is seldom used in commercial games.

There have been successful attempt on using artificial neural network in games, one example being the *Creatures* series where the player develop their own creatures, the interesting part is that the creatures are neural network driven and possess the ability to learn and evolve after a few generations - including learning a simple verb-object language or changing their behaviour through genetic algorithms. The game is commercially successful and the community is still generally active till today. More about it could be seen in the article written by [Grand and Cliff \(1998\)](#).

Yet, games utilizing machine learning are still a minority, even more so for RL. Although research has proved that it is definitely plausible to create strong AI that can easily outperform human players, such as AlphaStar reaching the Grandmaster level in *StarCraft II* ([Vinyals et al., 2019](#)). As of the date of writing, the writer is not aware of any games that use this technique and gain commercial success.

This is perhaps because of the fact that RL has some underlying problem which could be very difficult to solve, the two major problem is probably the balancing between exploration and exploitation, and long term credit assignment which is both crucial for RTS games AI.

6.4.1 | Long Term Credit Assignment

RL's biggest weakness is perhaps the fact that it is difficult for the algorithm to determine which crucial action or series of action will lead to the reward. To counter this problem, one approach is to assign credit frequently, this is known as *shaping rewards* as it shape the behaviour of the agent.

However, if we are not careful the agent might learn to exploit our shaping reward - instead of doing the crucial action that leads to the final reward, it might repetitively do a series of action that leads to a huge amount of shaping reward that eventually outweigh the final reward ([ClarkDario and Amodei, 2016](#)). This is known as *reward hacking*.

6.4.2 | Exploration and Exploitation

RL's core is to stochastically search through the action space given a state. Rewards will be given when the action is beneficial to the agent, and sometimes punishment will be given to detrimental action.

However, what seems to be correct *now* might not be actually the optimal behaviour, and might lead to huge loss after a period of time, and vice versa. To counter this problem, curiosity heuristic is introduced to encourage agents to explore the world when there is an unknown situation. This sometimes solves the problem very well, but also introduces its own problem, such as the agent procrastinating on taking any action because it does not know what to anticipate in the current situation for an extended period. This is also known as the TV-problem as, it was once reported that the agent got addicted to TV when exploring a maze (Savinov et al., 2018).

6.4.3 | Advantage of Reinforcement Learning

The advantage of reinforcement learning driven AI is that it does not require any pre-programmed behaviours. It only requires an interface that takes a group of input, which perform certain basic actions in the world, e.g walk in x-direction (as opposed to walk *to* a certain position), or interact with an object (as opposed to pushing a specific lever). This approach allows for emergent behaviour and tactics, which naturally makes the unit look less robotic.

There is already plenty of pioneers in related fields, but the most related one to my project is perhaps the one done by Baker et al. (2019), they have successfully trained AI that plays hide and seek in extremely effective manner, and the nature of hide and seek is remarkably similar to tower defense, in a sense both games require units' to move wisely to avoid danger and achieve the goal, in other words, TD's enemies is a combination of the hidiers and seekers - hiding from the turret and seeking for the player or the destination.

It is worth highlighting that their AI had developed various strategies, some of them even exploited the game's physics engine. Their report proved that this project is completely possible given enough resources and dedication.

6.5 | Reinforcement Learning Algorithms

In a later section, we will discuss an algorithm known as PPO which we used. However, before we can discuss PPO in details we have to first have some background knowledge about RL algorithms.

We will deliberately gloss over the details of various RL algorithms. This is because the detailed theory is beyond the scope of this dissertation, and they are often well explained in articles we referenced. We will instead give the intuition behind the algorithms and reference to the original paper whenever necessary.

In general, a typical RL training algorithms will be similar to figure 6.2. The agent will be put into an environment, given an input that represents the current state of the environment and the agent will then be required to output an action. The action will be rewarded by a certain "reward scheme".

The goal of the agent is to learn a policy (often notated as π), that will maximize the reward that the agent get from the reward scheme. There are two families of RL algorithms that aim to tackle this problem - *value-based* and *policy-based* algorithm.

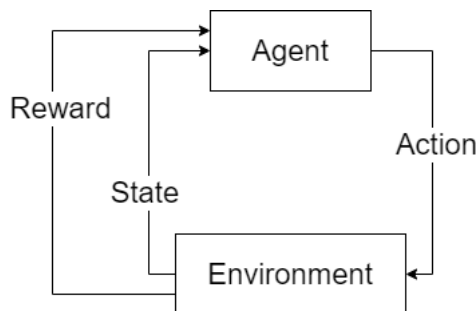


Figure 6.2: An overview of training process(feedback loop) in RL

6.5.1 | Value-Based Algorithms

To find the optimal policy π , value-based algorithms first find out "how valuable is it to be in a state s ". Which means before learning the optimal policy π , value-based algorithms first seek the optimal value function $V(s)$ by *exploring* the environment through acting randomly and sample the values of different state-action pairs. Quite naturally the value is often directly related to the reward one can get to be in a certain state. The aim of value-based algorithm is that it is relatively trivial to find the optimal policy given the optimal value function.

A typical example of value-based algorithm is Q-Learning (Sutton and Barto, 2018), where the value function also takes the action a as input to calculate the *Q-Value* which estimates the maximum cumulative future reward for taking an action in a certain state.

Value-based algorithms tends to be *sample efficient* as the agent can learn from any previous trajectory sampled from the same environment. However they often suffer from *curse of dimensionality* – the various problems one encounter when organizing and analyzing data in high dimensional-spaces. This is especially true when the input/output space is continuous, and hence would be computationally expensive to learn the correct value function through exploring the continuous space, such as the environment of our TD game.

6.5.2 | Policy-Based Algorithms

Policy-based algorithms try to find the optimal policy π *without* the aid of a value function. That is, instead of relying on the exploration of the environment and approximating the optimal value function, the agent will make actions according to a certain policy for the entire episode¹ of training, and only receive a cumulative reward after the episode is finished. The policy will only then be updated if it is strictly better than the previous policy, and hence the name - policy-based.

Most policy-based algorithms utilize gradient descent and hence also suffer from the disadvantages associated with it, such as the risk of over-stepping the global optima due to a large learning rate or getting stuck in a local optima when the function to be optimized is non-convex. A typical example of a policy-based algorithm is REINFORCE published by Williams (1992).

Policy-based algorithms are less sample efficient as, unlike value-based approaches, policy-based algorithms are not able to learn from the past. However, policy-based algorithms are often computationally simpler, especially in a highly stochastic environment or a continuous space, where approximating a value function is computationally expensive.

6.5.3 | Actor Critics

When there are 2 schools of algorithms, each with their own merits, one natural strategy is to combine their advantages, and hence actor-critic is introduced. In actor-critic methodology, the agent is trained in two parts:

Actor which strives to learn the optimal policy π and is responsible to make actions

Critic which evaluates the action taken by the actor by learning a value function, and suggest how the policy should be updated.

The general architecture of actor-critic methodology can be seen in figure 6.3. Where we can see that after an agent makes an action based on a policy, the environment will respond to it and produce a reward based on the reward function, the reward is then fed into the critic (the value function), and gives reward through a TD (Temporal difference) error function – TD error describes the expected reward changes by moving into a new state with the action taken by the agent in the current timestep. After the reward process, the agent will again receive the input state and output the action. This process continues and the agent eventually learns the estimated optimal policy of the environment.

¹An episode in RL refers to a period of time between the start of a training round and the end of training round. Imagining playing go, the entire game of go will be an episode in RL.

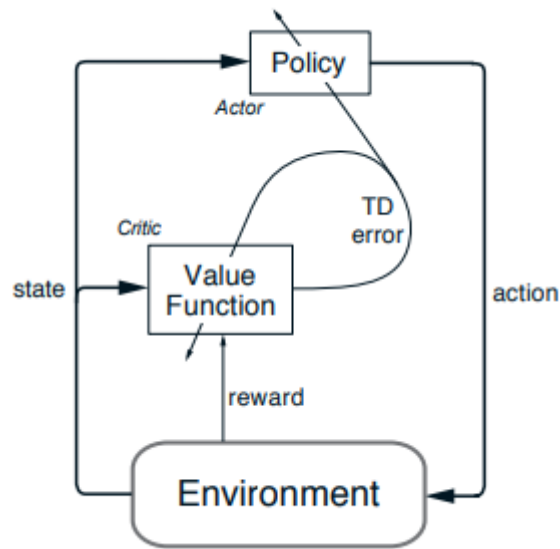


Figure 6.3: Basic architecture of actor-critics(Sutton and Barto, 2014)

The perhaps most important improvement on actor-critics is the Advantage Actor Critic (A2C) model, where instead of using a value function $V(s)$ to aid training, we use an *advantage function*, defined as $A(s, a)$, that output "the advantage of taking an action over the other action on average", hence the name "advantage" function.

6.5.4 | Algorithm that I Used – Proximal Policy Optimization

PPO is in a nutshell, a variant of A2C released by OpenAI that aim to achieve high and reliable performance with low computational complexity, with as little hyper-parameter tuning as possible(Schulman et al., 2017).

We will be using PPO throughout the entire development process. As PPO typically requires less hyper-parameter tuning, which requires a huge amount of time and computational power which we do not have.

As previously mentioned in section 6.5.2, most policy-based algorithms depends on an optimization method known as *gradient descent*, which is either too fast and can easily falls in to local minima with a large learning rate, or painfully slow to train with a low learning rate(Ruder, 2016b). PPO takes inspiration from Trust Region Policy Optimization (TRPO) and attempts to prevent the agent from making drastic changes to the policy. PPO achieve this by first measuring the difference between the old and new policy based on their KL divergence, and clipping the advantage function for any new policy that is significantly different from our previous policy.

Clipping the advantage function is a rather *soft* constraint – the agent merely has no preference for changing to a more different policy over the other, the agent is not forbidden from choosing a very different policy, as such the agent will inevitably make bad decision once in a while, yet PPO successfully strikes the balance between simplicity and effectiveness, and out-performs other state of arts alternatives in many environments (Schulman et al., 2017).

6.5.5 | Curriculum Training

During our development of RL agent, curriculum training is one of the technique that we used extensively, and has gone through couple of changes throughout the course of development.

Curriculum training is a technique that based on the believes that some task is too difficult for the agent to learn from scratch, and it is easier to learn the task if we simplify it at first to guide the agent into the right direction. After that we gradually increase the difficulty of the task until it is identical to our ultimate goal. We believe that it will be easier for the agent to converge to the optimal policy through this process of learning a *slightly* harder problem every level. To put it simply, if we want the agent to run, we first teach the agent how to walk.

In RL, this is achieved by changing the environment according to the agent performance. When the agent perform consistently well enough, it will be progressed to a new "level", the higher the level is, the harder the task will be, and more similar the scenario will be with the actual gameplay. Curriculum training is commonly used in complicated RL task (Gupta et al., 2017).

AI Development through Reinforcement Learning

We are utilizing Unity ML Agent library for our [RL](#) development, thus unlike in [section 4.1](#) the focus in this chapter will be on the methodologies employed and the reasoning behind them, and how it interacts with the environment.

Unlike our previous discussion of the gameplay, I will first elaborate on our initial attempt, and then based on the problem we faced we will discuss about how we systematically introduce new techniques and improve our training process. As with many Machine Learning ([ML](#)) algorithms, before getting good results for a particular variant, a good deal of hyper parameter exploration and tuning was needed. However, as the results of this are largely self-evident and not terribly interesting, we have left this out to focus on key results.

Note Unity ML Agent library will only handle the implementation of [PPO](#) and [GAIL](#)(which is not used extensively) in our development, the training process, generation of input, interpretation of output format, and design of curriculum will all be implemented by us. In particular, I had to extend on Unity ML Agent library such that it allows another format of input that I used for our [RL](#), which will be discussed in detail in a later section.

7.1 | Overview

In our training, the agent will be given an input that represents the surrounding environments, and is required to output two values X and Y , based on a *policy*. The output values will then be used to control the movement of agents. The ultimate goal of the agent is to reach the target's position, while avoiding obstacles and turrets when necessary.

After each episode of training, the environment will reset and the agent will evaluate the efficiency of its current policy, based on the performance of the policy, the agent will then decide whether or not to adapt the new policy. Hence the agent will be iteratively better at the task by using an improved policy over time.

On top of basic reinforcement learning, we also make heavy use of curriculum training, which are covered in chapter 6. Figure 7.1 shows how the averaged reward is feed into the curriculum controller, which output the current "level" of the training curriculum and hence controls the state.

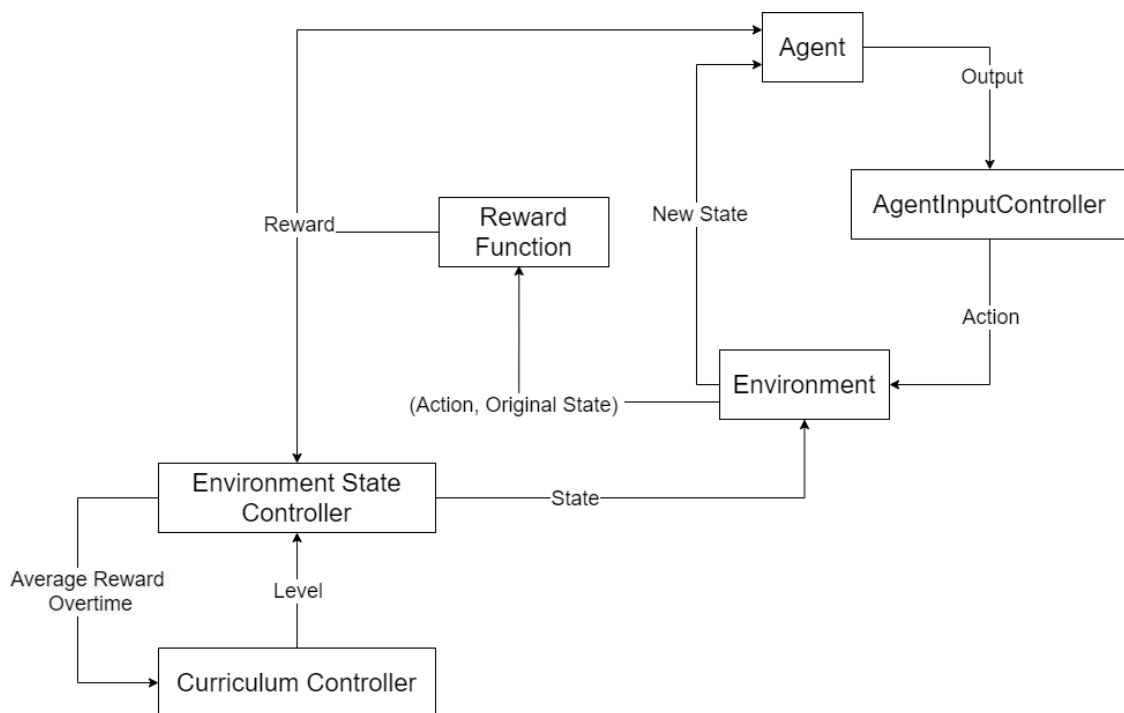


Figure 7.1: Brief demonstration on how our training process is structured

7.2 | Initial Settings

In this section, we will discuss the initial configuration of our training process. All of these configuration is improved in future iteration, we will discuss the improvements in depth in later sections.

Category	Value
Collide with Obstacle	-0.5
Agent Killed	-0.15
Approaching Target	$0.1 \cdot \frac{\mathcal{D}(\mathcal{A}_i, \mathcal{A}_{i-1})}{\Delta_{max} \mathcal{A}}$
Roaming Punishment	-0.025
Contact with Target	1

Table 7.1: Reward received by the agent after making any decision in different circumstances

7.2.1 | Reward Function

The reward function remains the same for the entire development process. In a nut shell, in every step, the agent is rewarded by the reduced distance between itself and the target, and is punished whenever it is killed or when it collide with obstacles. To encourage the agent to make contact with the target as soon as possible, it is also rewarded when it made contact with the target, and is punished by a small amount for every step during the episode. Note to prevent reward hacking, the agent will not receive a reward from approaching the target if the current distance between the target and the agent is not shorter than the shortest distance between the agent and the target during the entire episode, this way the agent cannot yield reward by going forward and backwards. Table 7.1 demonstrated the reward function in details.

7.2.2 | Output Format

There is only three possible value for both X and Y output from the RL agent – 0, –1 and 1 as demonstrated in figure 7.2. The output will be neglected if it is 0 – representing the agent not wanting to move. Both 1 and –1 will be interpreted as control signal from the agent and will cause the agent to either turn or walk forward/backwards.

7.2.3 | Initial Input Format

Our main requirement for the RL-driven agent is to navigate in a highly dynamic world, where arbitrary numbers of turrets and obstacles can be placed at anytime during gameplay. This has bring a heavy limitation on our input – I cannot rely solely on vector input(an input format that is simply an array of values) as it is fixed in size, and hence

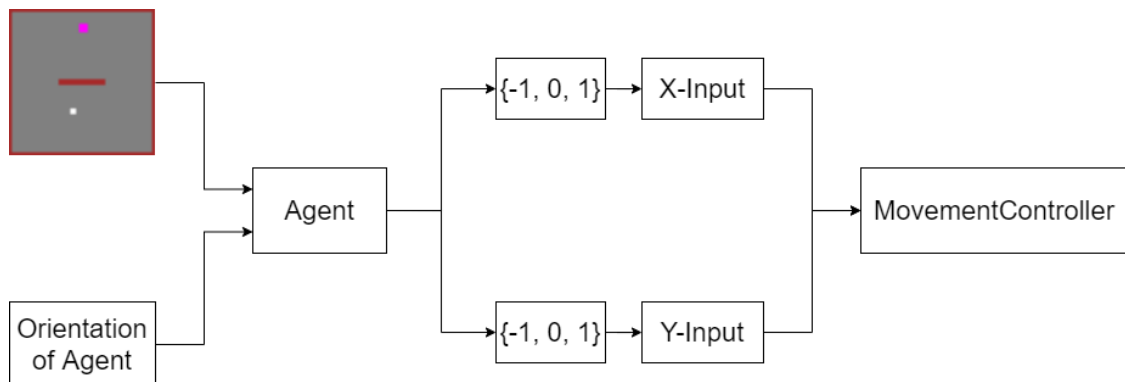


Figure 7.2: Demonstration of how the initial input and output is structured.

cannot encode an arbitrary amount of turret and obstacles. I am forced to use a *visual input* to represent the world.

However, visual input tends to be very noisy, this is especially true in a game environment where particles and animation are everywhere to make the game visually appealing. Thus a need of "a visual representation that is not noisy" arises. To achieve this, I have made some slight extension to Unity-ML library, to enable a customized *image* input. This extension allows me to use input as image.

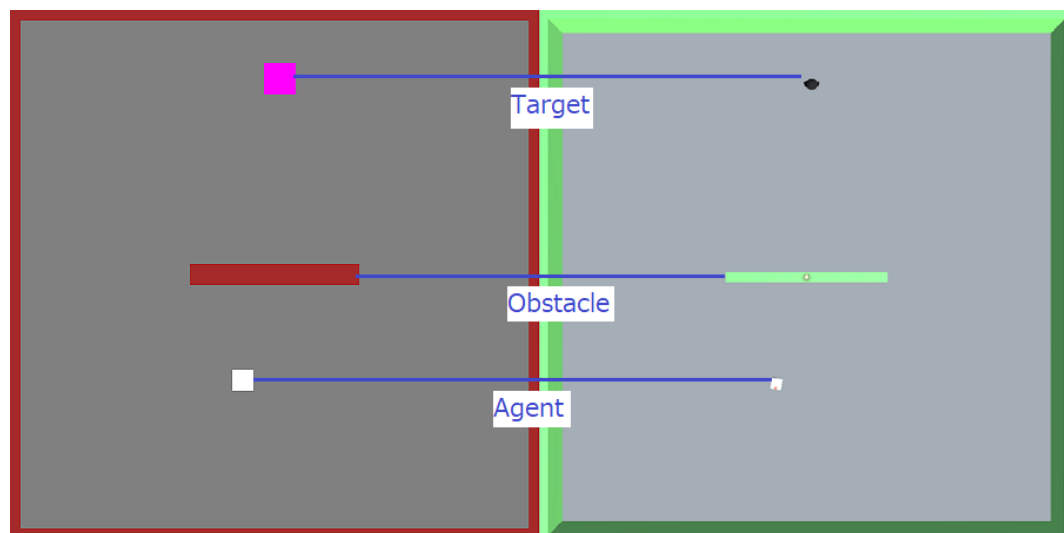


Figure 7.3: Demonstration of conversion between the top-down view of the environment and the input image in the first iteration

As figure 7.3 shows, we encode the world by taking the coordinate of all entities in

the world, and draw all entities of the same type in the same color. Hence the agent will be able to realize what each color represents and make decisions respectively. Note this is essentially a top down view of the environment, but with all the visual effects(shadows, particles, etc.) stripped off.

However the image input alone did not encode information about the agent's orientation – the agent needs to be aware of the direction it is facing, otherwise it could not possibly approach the target. To compensate this, I decided to use a mix of the image input *and* vector input, the vector input will contains one element, a 0 – 1 value representing the normalized rotation in y-axis of the agent, this is demonstrated in figure 7.2. I believed that this will provide enough information for the agent to approach the target.

7.2.4 | Initial Curriculum

In the initial curriculum, our agent and target were randomly spawned in the environment with a constraint that the distance between them should not be higher than a certain constant D . As the agent progress into new levels, D increases such that the agent and the target can be potentially further away from each other. The target size is also increased in early stages.

The idea behind this is that it is that approaching a target closer to you(or larger) is an intuitively easier task compared to approaching a target further away from you. I hoped through simplifying the task in early stages, it would increase the chance of the agent stochastically finding its goal of approaching the target. Thus helping it converge to the optimal solution.

Level 0 to 3 are also laid out such that there are no obstacles in between the target and the agent. This is because we believe that it should be possible to split the task of approaching the target and avoiding obstacles into two – the task to avoid the obstacle, and the task to approach the target in a straight line. I believed by introducing one task at a time the agent should be able to converge quicker.

An important note is that using our original curriculum, we never reached a state where our agent performs well enough that we can determine that it could function in the true environment, and hence the task is always simplified in our original curriculum, as shown in figure 7.6.

7.2.5 | Results

The agent did not perform very well – in fact it barely learn to approach the target even without any obstacles, most of the time it stutters and appears to be moving

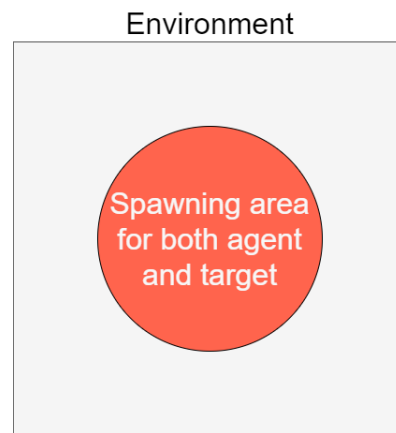


Figure 7.4: Demonstration of the environment is set up in the first level



Figure 7.5: Demonstration of the environment is set up in the final level

randomly, although with a slight tendency of moving near the agent. Figure 7.7 shows the performance of the agent in the initial attempt.

7.3 | Improvement in Input Format

Input format is changed significantly and yield very significant improvements. In the following sections we will discuss the various changes we made in the input format and how it changes the performance of agent and the robustness in the training process.

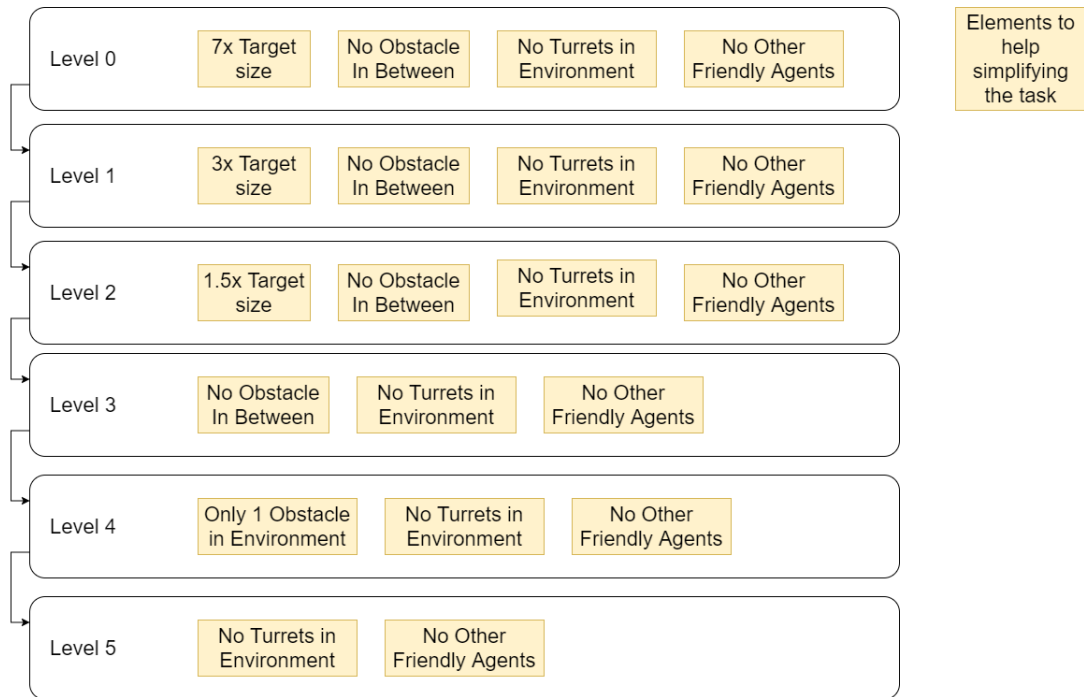


Figure 7.6: Demonstration of general structure of the original curriculum

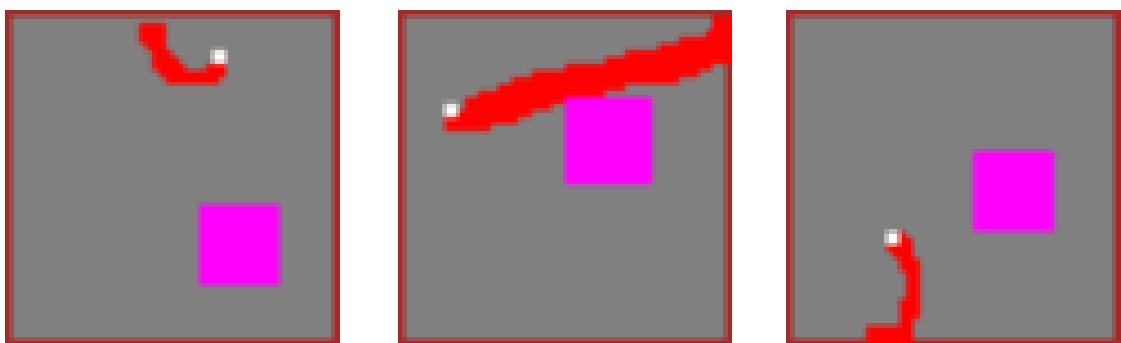


Figure 7.7: Performance on the initial iteration, note even with enlarged target size, the agent only displays a very weak tendency to approach the target

7.3.1 | Rotate Image to Encode Agent's Orientation

After the previous iteration, I have been trying to fiddle around for quite sometime, and discovered that it was never explicitly mentioned that Unity ML could actually handle a combination of image and vector inputs. There is no code example related to it and is not mentioned in their documentation either.

7.3.1.1 | Hypothesis

We realized there is no information in the Unity ML documentation regarding if image and vector observation can be used at the same time, we hypothesise that it is not supported, and this implies that we have to use image to somehow encode the orientation of the agent.

7.3.1.2 | Methods

We test our hypothesis by only using image observation, and if the training improves when we only use image observation despite the increased input complexity, we conclude our hypothesis to be correct.

To encode the rotation using the image and the information environment, I decided to rotate the entire image according to the agent's orientation. This operation however, is more complicated than I originally imagined, it brings 2 significant implication to all our further iterations. Enlarged dimension in images and loss of information.

Enlarged Dimension: The image has been a $50 \cdot 50$ image so far, with each pixel representing $1m^2$ space of the environment. However, if we were to rotate the image according to the orientation of the agents, the image must be enlarged to cope with the necessary operation.

Using the Pythagoras's theorem, the new image will need to be of size $\sqrt{x^2 + y^2}$. Thus the rotated image will be of size $71 \cdot 71$ pixels, effectively doubling the size of input. The increased size of input means that it will take a longer time for training due to an increased amount of data to be processed, besides it also means that it can potentially be harder for the agent to learn, as the input space is more complicated. Figure 7.8 demonstrated how the image size will be increased.

Loss of Information: The rotating process will inevitably destroy some of the information in the process, for example, if the target size is too small, the target can be completely "wiped out" after rotating due to the lost of information.

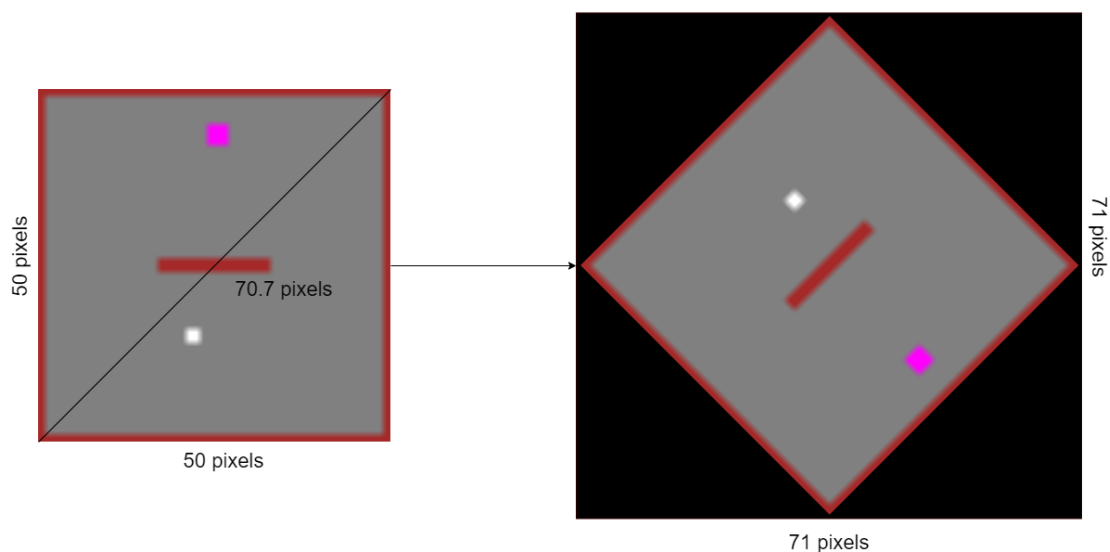


Figure 7.8: Demonstration of how rotation will increase the image size

To counter this, I had to make sure the agent and the target (smallest entity of the entire environment) occupy at least one pixel in the final image. However, this does not solve the problem completely as the result image is now inherently inaccurate, due to the drawn pixel might not be representing the actual position in the environment. One way to solve this problem is to further increase the dimension of the image, hence increasing the precision of representation of the environment – but this would again further enlarge dimension on each axis.

7.3.1.3 | Results

The performance of the trained model is still, not very satisfying. The agent still struggles to approach the target even without any obstacle.

However, in figure 7.9, we can see a significant improvement at the end of training, the cumulative reward has significantly increased, and the episode length being significantly decreased – both of these implies that the agent is slowly approaching the target.

I thus assumed that the rotation of image is providing sufficient information to the agent to start learning, however, it is not acceptable to further increase the maximum number of steps used to train the agent¹ due to limited computation resources, training

¹The maximum amount of steps refers to "how many decision" the agent can make before the training is finished, the larger the number, the more training the agent will receive, and typically the better the performance. This of course also increases with the training time.

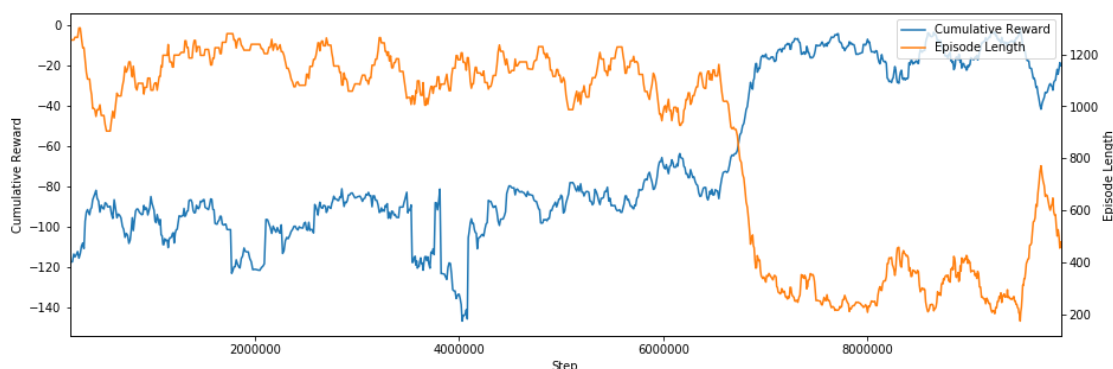


Figure 7.9: Cumulative reward and episode length of Agent During Training, note all figure related to training metrics smoothed are smoothed by taking the mean value of every 25 episode, otherwise the graph will just be too noisy to be of any use.

with 10.0×10^6 steps is roughly taking 12 – 18 hours. Further increasing the amount of steps for training is simply not plausible – I had to come up with a way such that the agent can learn in a more sample efficient manner.

Although the performance is not very satisfying, the fact that the performance is improved gives us the confidence that our hypothesis is correct – otherwise there should not be an improvement given the significantly more complicated input space.

7.3.2 | Image Translation

Following the discussion with my supervisors, we decided to add a translation operation to the image, such that the agent is always positioned in the middle of the image.

7.3.2.1 | Hypothesis

We hypothesise that the agent will learn easier if the relative position instead of absolute position is encoded through our image. This echoes with what is recommended by Unity ML Agent library as well (MLB, 2019).

We hope by using relative position, we can achieve a semi-invariance in the image input – no matter where the agent and the target are, if the target and the agent are close to each other in the environment, the target will always be near the center of the image. We are assuming that having this invariance will make the training significantly easier.

7.3.2.2 | Method

We decide to encode the relative position of the agent by translating the image such that the agent is now *always* in the center of the image and is no longer drawn anymore. This process is shown in figure 7.10.

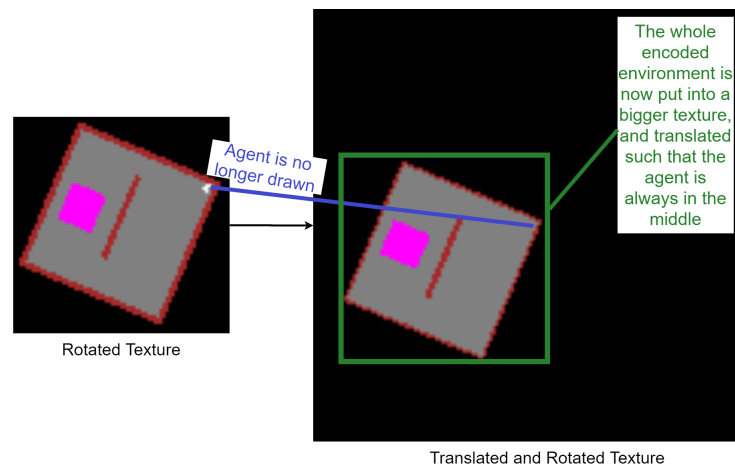


Figure 7.10: Demonstration on how translation of the image works

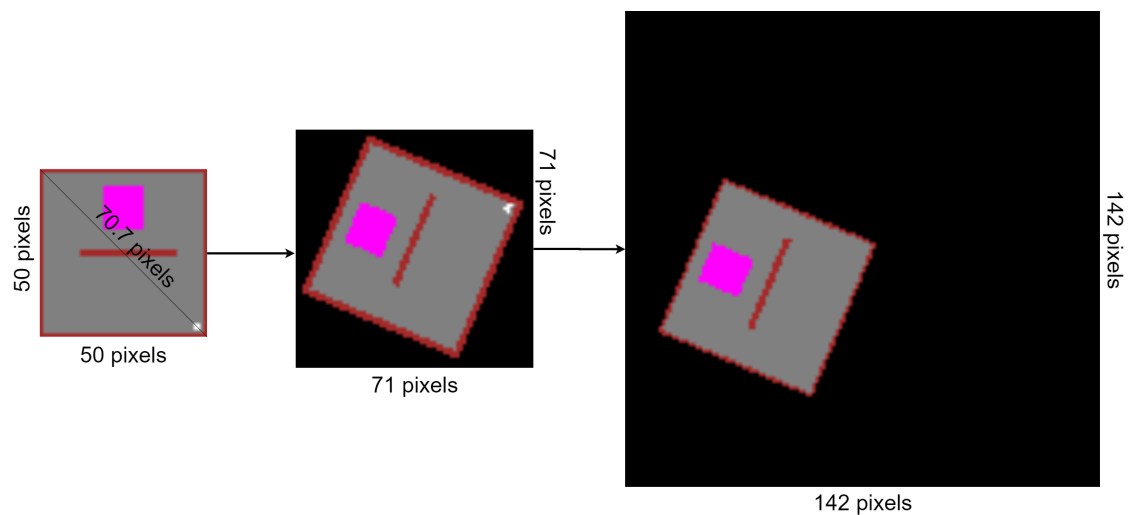


Figure 7.11: Comparison on the image size with each operation, note after both operation, the image size is eight times larger than the original one

Although this method successfully encode the relative position of the agent, this also means the image will be further doubled in each dimension (meaning four times in

size). Implies that the issues mentioned in section 7.3.1.2 will be further exacerbated. The image size comparison can be seen in figure 7.11.

7.3.2.3 | Results

The improvement is significant – the agent quickly past through the first three stages, but is however stuck on the fourth stages, which is also the first stage where the agent has to circumvent an obstacle.



Figure 7.12: Agent successfully traversing when there is no obstacles

The agent at this point, can effectively approach a target at a straight line as long as there is no obstacle between them, as demonstrated in figure 7.12.

7.3.3 | Simplifying Color Encoding

At this point, we assumed that our training of RL is going in the right direction. We try to build upon it again by simplifying the color encoding of image.

7.3.3.1 | Hypothesis

We hypothesise the difference between any color that is has value in more than one RGB channel is too subtle to be distinguished by the *simple* visual encoder that we use to encode our image observation(the other alternatives include ResNet(Espeholt et al., 2018) and Nature's CNN(Mnih et al., 2015), which is significantly more complicated and require more computation resources), and will hence further slows down the learning process.

7.3.3.2 | Method

We test our hypothesis by changing our color encoding such that every entity is either red, green, blue or black. This of course, will make encoding of environment a lot more limited. However given that we are at a very early stage where we only wanted to make

a minimum viable product and build upon it afterwards, we decided to proceed with this limited encoding.

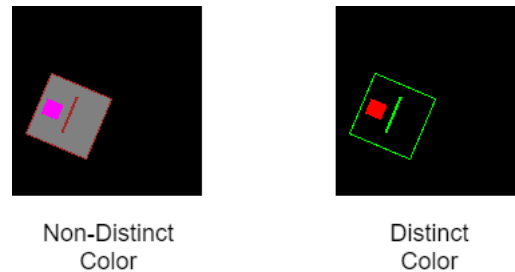


Figure 7.13: Demonstration on how the input image is transformed to use distinct color

The limitation can be easily seen in figure 7.13, where we are encoding both null areas² and the floor with black³.

7.3.3.3 | Results

The results is very strange – it is the first time that it breaks through to level 4 during training process. Meaning it is, in theory, able to circumvent obstacles.

However in practice, it does nothing but rotating, and displays no interest in moving forward. We are unsure of what is happening. We suspect this to be a result of randomness and will further discuss in a later section.

7.3.4 | Further Simplification – Grey Scale Color Encoding

Based on the training metrics using distinct color encoding, we want to further simplify our image input to both speed up the time to train each episode, and to make the training itself easier such that the agent can learn faster. We decided that we can further simplify our image input by using grey scale image, instead of all 3 channels for full RGB values.

7.3.4.1 | Hypothesis

We hypothesise that by using a grey scale image to encode the environment, not only the agent would be able to learn faster due to the decreased input complexity, but also the training time will be significantly decreased as there is less to be processed.

²Null area are positions that is out of the environment

³Blue is reserved for turrets, however our agent have not proceed far enough to reach a level where turrets exists

7.3.4.2 | Method

The advantage of using a grey scale image instead of a image with all RGB channel is that unlike our previous input where we requires three float value to represent one pixel, now we only need one float value for one pixel. As a result, the input size is effectively decreased by a factor of three.

However, this does implies that we have less freedom when we were to encode different entities. Besides, we also suspect that it is possible that the agent might be confused by the simplified image, as there is less difference between different entity in our new encoding.

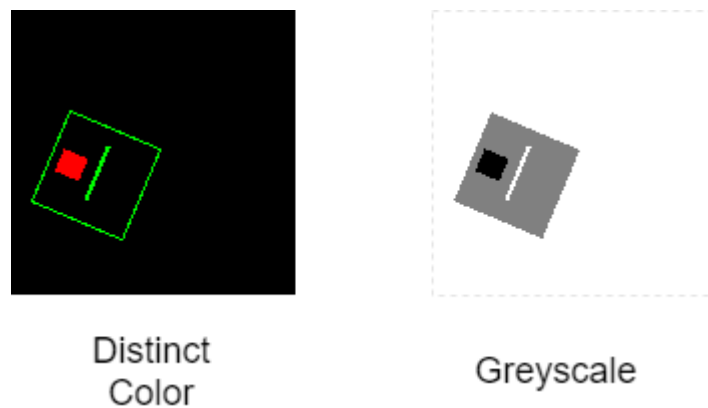


Figure 7.14: Demonstration on how the input image is transformed to grey scale

7.3.4.3 | Results

The results is satisfactory, we can see in figure 7.16 that the performance is similar to our previous iteration in terms of the episode length during training in the early phase, and outperforms the previous iteration by a huge margin at the end, the training time is also shortened significantly as shown in figure 7.15.

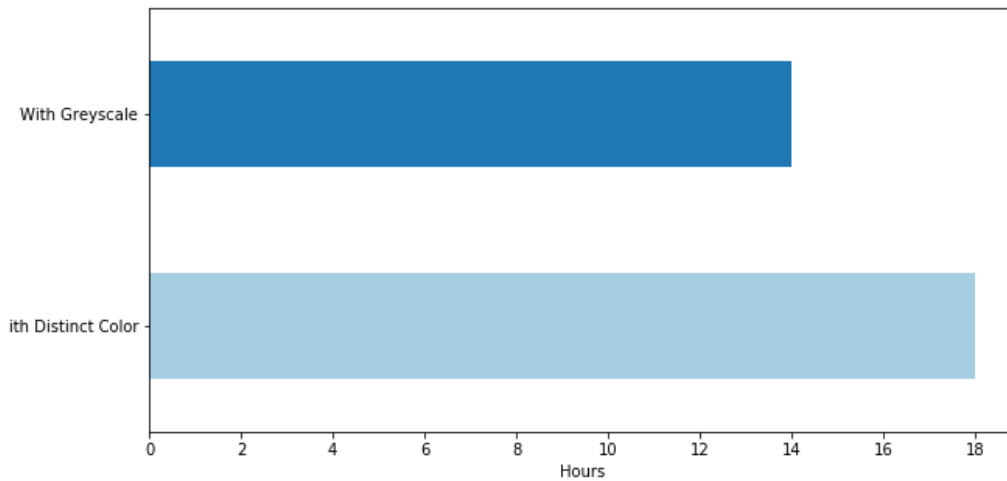


Figure 7.15: Rough comparison on training time with different color encoding

Besides, this time our agent is able to navigate around simple environment, unlike our previous models where it just rotate in an apparently random manner. We will discuss about the performance in the section 7.3.4.4 along with other findings.

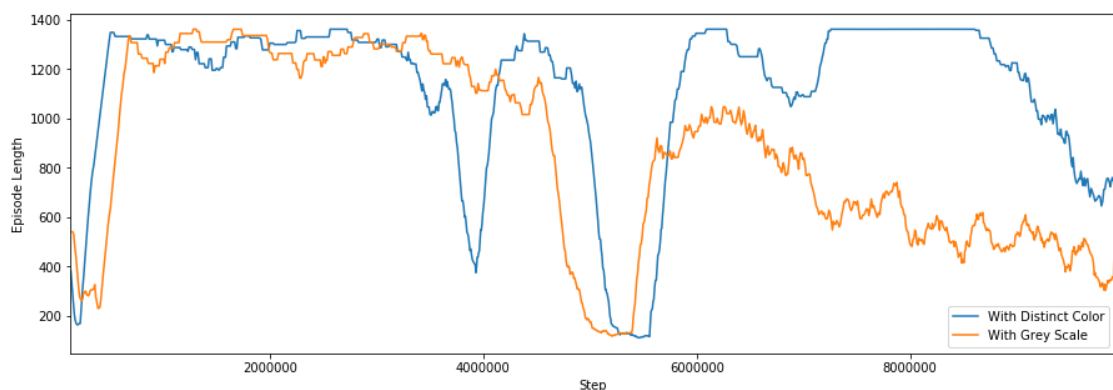


Figure 7.16: Comparison on episode length for when using different color encoding

7.3.4.4 | Remarks: Randomness in Training

I re-ran this iteration for debugging purpose using the same configuration. This provided a dramatic demonstration of the role that randomness plays in RL, something we shall

elaborate more in a section 7.5.1, for now we will focus on the findings on this particular scenario.

I first discovered this phenomenon by inspecting the training statistics, as shown in figure 7.17, during the second run, the episode length is consistently lower than that of the first run until around 7 million steps, and even after that, the episode length of the second run quickly decreased and surpass the first run in the end.

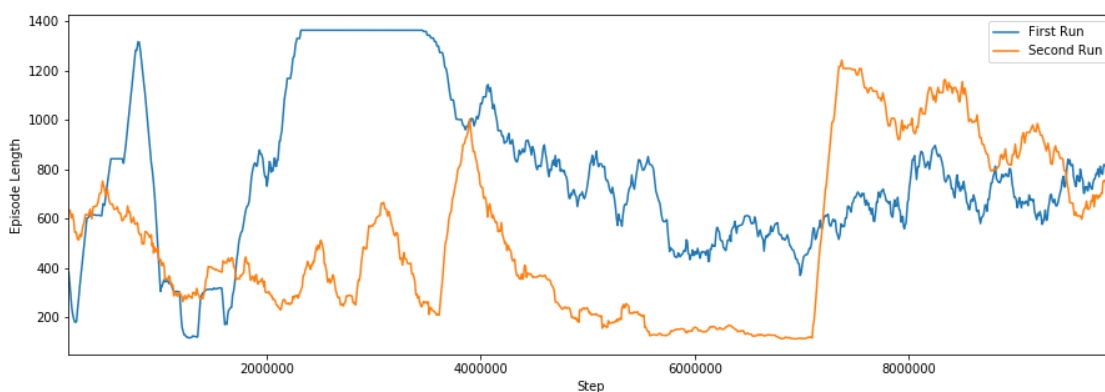


Figure 7.17: The episode length of iteration 6 in the first and second run

Such a big difference drives me to further examine the actual performance of the models, and against my original expectation, using the same testing environment, the 2 models has a drastically different behaviour. By examining figure 7.18, we can see that the first model fails to approach the target and stuck in a loop very soon after it departs, whereas the second model was able to find a proper path towards the target in a more or less smooth manner.

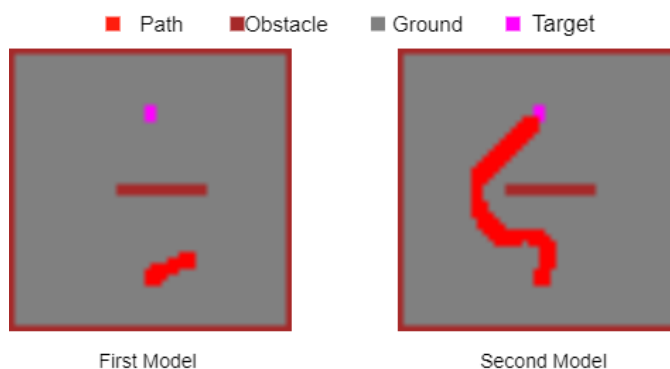


Figure 7.18: Demonstration of path taken by the agent using the first model and the second model. Both trained with the same configuration

It is the first time that randomness has brought to my attention – I used to assume that after 10 million steps, randomness should be more or less "smoothed out" by the large amount of training data it gets. However this results makes me realize that it is not necessarily the case, this also suggested that there should be methods to increase the stability of training outcome.

7.3.5 | Conclusion On Input Format

Table 7.2 shows the performance and their corresponding training configuration. We can see that the agent improves with every technique we apply to our input format, iteration 5 is the only exception – I suspect this to be the result of randomness as discussed in section 7.3.4.4, as we can also see that in figure 7.16 the training metrics in iteration 5 is relatively close with that of iteration 6 at the end of training.

	Vector as Rotation	Image Rotation	Image Translation	Distinct Color	Grey Scale	Performance
Iteration 1	✓					0/2
Iteration 2		✓				0/2
Iteration 3		✓	✓			1/2
Iteration 4		✓	✓	✓		0/2
Iteration 5		✓	✓		✓	2/2

Table 7.2: Performance of agents with different configuration, 0/2 for no observable behaviour, 1/2 for efficient with no obstacles, 2/2 for acceptable with simple environment

7.4 | Improvement in Curriculum

After we improved our input format, I decided it is better to move on to the other aspects of training configuration, as it is very possible that there are some quick improvements we can make in those areas.

After investigation, I discovered that another factor that affects the performance of the agent significantly is the curriculum, we will elaborate on how we improved on our original curriculum by only making slight changes, yet yielding significantly different result.

7.4.1 | Introduce Obstacles in the Early Levels

In section 7.3.4.4, we have discussed the role randomness plays in training process. It seems to us that there is a need to reduce this randomness as it significantly affects our

model's performance. We hypothesise that the randomness is a result of our current curriculum misguiding the agent into a local minima.

7.4.1.1 | Hypothesis

To gain more insight on what is happening in our previous iteration mentioned in section 7.3.4, we inspected some of the training metrics in the first run of iteration 6 and gained some interesting insights.

We observed that our agent passed level 0-3 rather rapidly, the agent was already able to navigate to the target in a reasonable manner spending less than 1/5 of the total training time. However upon reaching level 4, both curiosity forward loss⁴ and cumulative reward decreases rapidly as shown in figure 7.19. Curiosity forward loss never increase afterwards, and although cumulative reward eventually increased, the agent was never able to learn enough to circumvent the obstacle effectively.

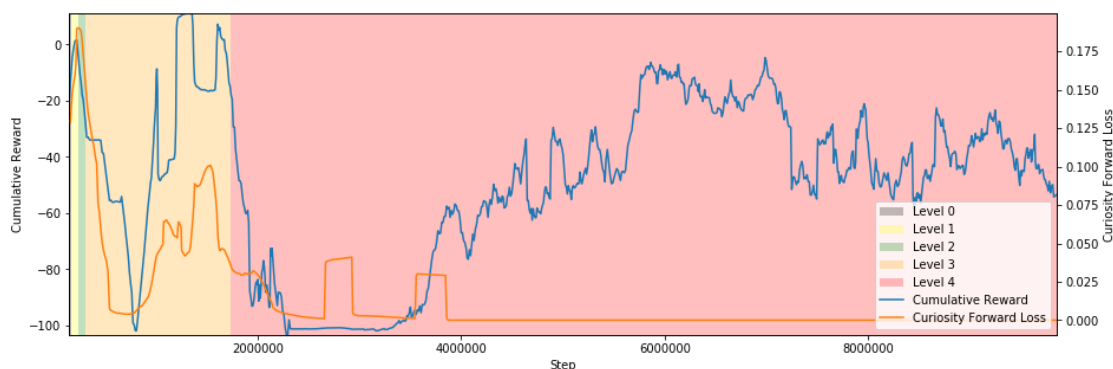


Figure 7.19: In level 4, the agent received a very low extrinsic reward and curiosity forward loss for extended period of time.

The fact that the curiosity forward loss stay low(close to 0) is very surprising – this indicates that the agent *expected* how the environment will change, but was unable to find the proper action to take to approach the target. The consistently low cumulative reward in the later phase also explains the behaviour of the agent stuttering⁵ when there is an obstacle in between the agent and the target.

⁴Curiosity forward loss indicates how "surprised" the agent is by the environment changes resulting from the agents action. The larger it is, the more reward the agent receives, and hence encouraging the agent to explore.

⁵This is where I think it will be nice to have a stutterness metrics.

Based on these metrics, we assume that during the training process the agent was misguided to not approach the target whenever there is an obstacle in between them, instead of attempting to circumvent the obstacles.

Thus we made a hypothesis that the task of approaching the agent while avoiding obstacles may be a task that cannot be further separated into multiple sub-tasks.

7.4.1.2 | Method

In order to test our hypothesis, we carried out a test by training under a new curriculum based on the assumption that the navigation task cannot be divided. We modified our original curriculum so an obstacle *always* exist between the agent and target, but not necessary blocking the agent from the target. The other aspect of the training remains mostly unchanged. This is demonstrated in figure 7.20.

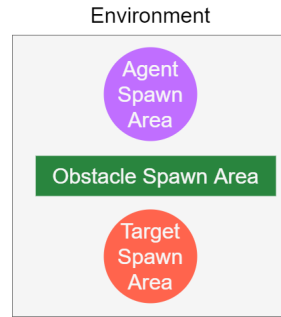


Figure 7.20: The spawning area of various entity in our testing for the new curriculum

If our hypothesis is correct, our new curriculum shown in figure 7.21 will yield approximately similar performance to what we have from section 7.3.4, but in a more stable manner. It is important to note that at this point, we are not aiming for a strictly better performance, but rather achieving what we already have in a more reliable way

Due to limitation on computation power, we were unable to train for 10.0×10^6 steps like our other iteration when we are testing – instead we have only trained for 2.0×10^6 steps, yet this should be sufficient to determine whether our assumption is correct.

7.4.1.3 | Results

With our new curriculum, all models demonstrated ability to circumvent obstacles, and displayed a strong tendency to approach the target as shown in figure 7.22. Although not every model made contact with the target in this particular experiment, the path of all agents trained in the new curriculum hugely overlap, implying they are more stable in the learning outcome.

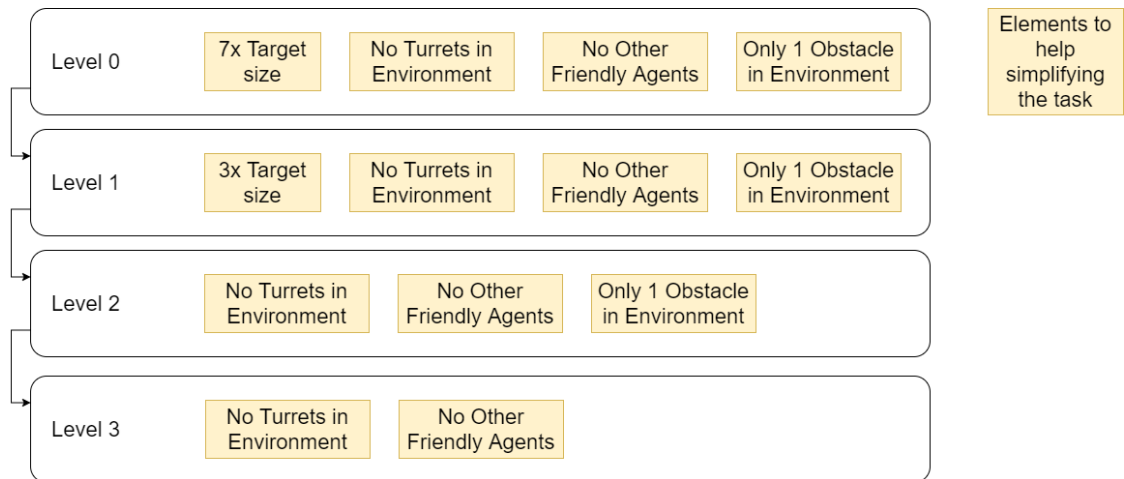


Figure 7.21: New curriculum that does not attempt to split the navigation task into sub-tasks

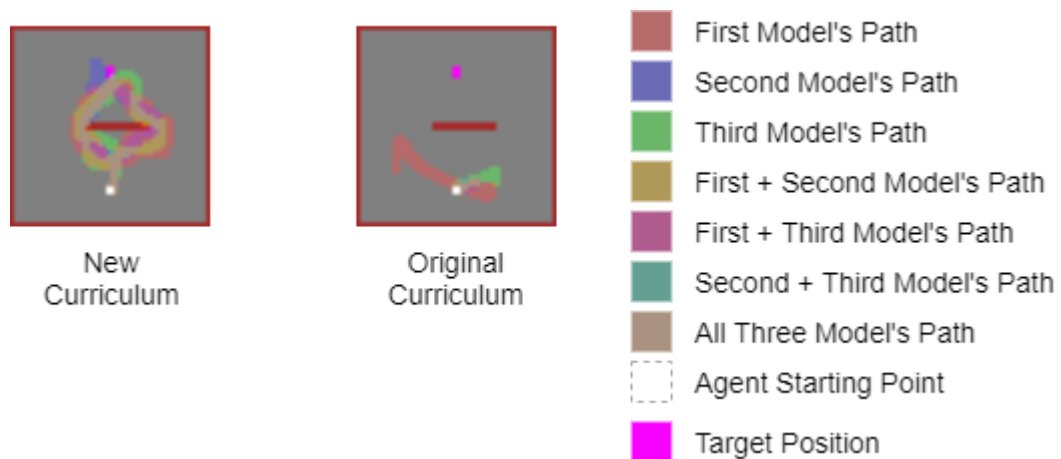


Figure 7.22: Paths taken by all 6 models on the same environment, significantly more overlap on path token by models trained on the new curriculum.

In contrast, when training with our old curriculum, the path taken by different models rarely overlap, showing a high variation in their behaviour. Besides, in terms of performance, none of the models were demonstrating any ability to circumvent the obstacles, and only demonstrated a very low tendency to approach the target when obstacle exists.

The result proves that our hypothesis is correct – it is highly possible that the navigation task cannot be split into sub-tasks of "avoiding obstacle" and "approaching target", and the switch of curriculum helps stabilizing the training process.

After we have proven our hypothesis, we decided it is worth doing a longer training run on our new curriculum. The results yield from training for 10.0×10^6 steps using the new curriculum is satisfactory – the agent can finally navigate in a more complicated environment.

The agent in general behaves in a smoother manner with improved performance. It is able to navigate in various rather simple environment, as shown in figure 7.23.

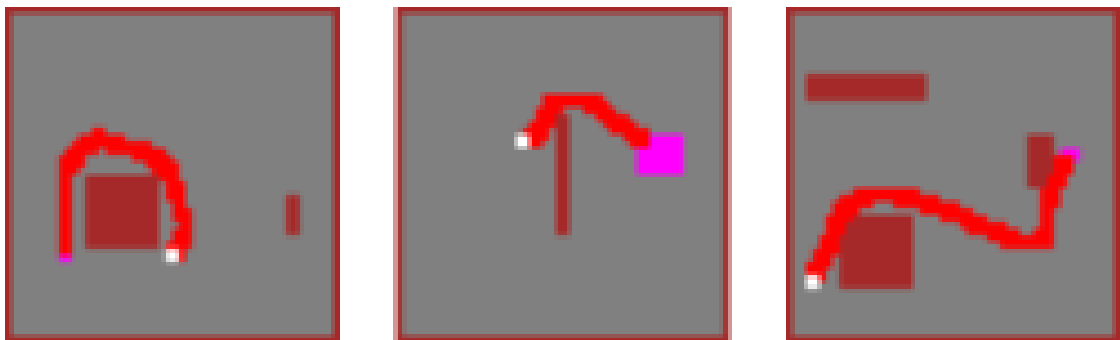


Figure 7.23: Environment where the agent successfully navigate towards the target

However, there are still many environments where the agent will fail to find a path towards the target and start stuttering. although this problem will be largely mitigated during gameplay as the target(player) will be constantly moving, this is still clearly not good enough for our goal.

After a more thorough investigation, we discovered that the agent seems to grow "reliance" on the existence of obstacles. As obstacles appear most of the time, the agent seems to have mistakenly decided that circling the obstacle will lead to a rewards. This is particularly apparent in figure 7.25. This circling behaviour also exist in the first path shown in figure 7.23, as the agent take a roundabout route instead of the shortest path possible.

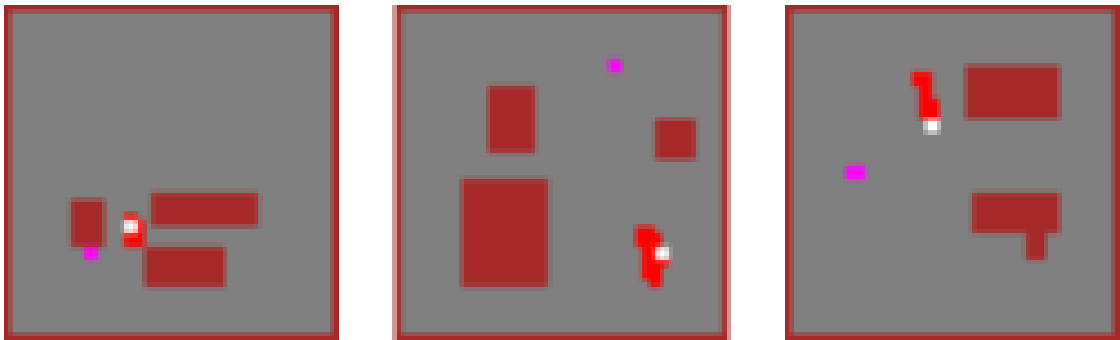


Figure 7.24: Environment where the agent failed to navigate towards the target

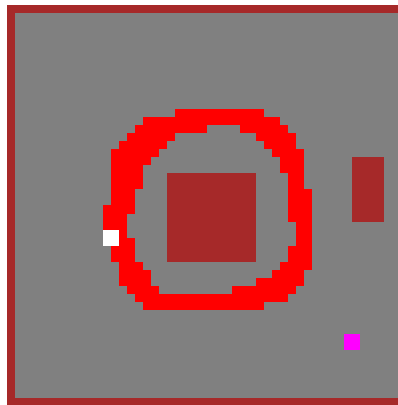


Figure 7.25: Agent circling the obstacle, even if the optimal path towards does not require the agent to do so.

Regardless, at this point the agent is finally *functional* – it can be in theory to be used in our game, although there is plenty to be improved.

7.4.2 | Randomize Obstacles in Early Level

Although our agent is functional, it is clear that it is still far below our expectation. In particular, the circling behaviour can be very obvious to player that the agent is not intelligent. My goal has hence become to resolve this circling behaviour.

7.4.2.1 | Hypothesise

As we have previously discussed, we hypothesise that the agent is growing an unnecessary reliance over the existence of obstacles, which leads to the circling behaviour. I now

further hypothesise that the reliance is caused by the fact that the obstacle is always in between the agent and target in initial stages, and misguided the agent to believe that circumventing the obstacle will *always* leads to a reward.

7.4.2.2 | Method

To test our hypothesis, we designed a new curriculum such that obstacles are spawned in a way similar to the actual gameplay starting from the first level, essentially we no longer simplify the task by simplifying the environment through only introducing one obstacle in between – now the agent will navigate in the shrunken real game environment right from the get go, except the target size being increased in early levels.

We conclude that our hypothesis is correct if this curriculum removes the circling behaviour.

7.4.2.3 | Results

The new curriculum yields a significant improvement in performance. In terms of episode length shown in figure 7.26, the episode length is consistently lower when using the new curriculum, indicating that it is able to navigate towards the target in a shorter amount of time during training.

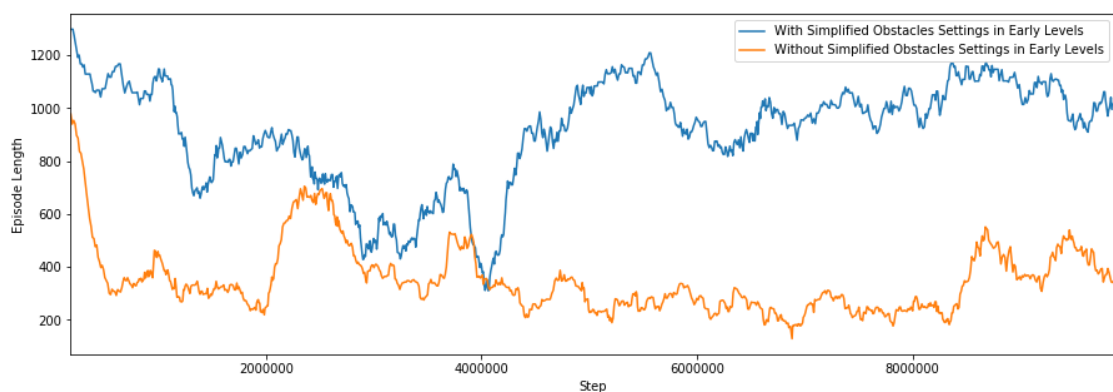


Figure 7.26: Comparison on episode length – with and without simplified obstacles settings in early levels

Using our previous test suite, it is not immediately obvious that the new curriculum surpass our previous ones – in fact the performance might even appear to be worst. This obviously contradicts the training metrics, and after having a closer we found out that the new agent is confused when obstacles are too *thin*. This can be seen in figure 7.27,

where our new agent fails in test 2 and 3 with thin obstacles, but performs smoothly when the obstacles are thicker.

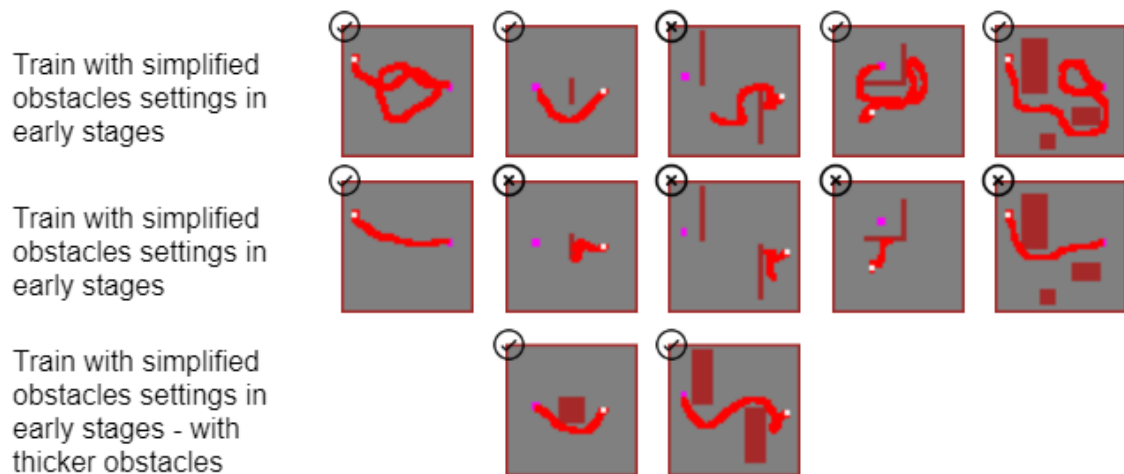


Figure 7.27: Comparison between path taken by our agent trained in new curriculum and old curriculum

Regardless of this caveat, it is clear that the circling behaviour is removed, this is particularly obvious in the first and fifth test, where the old agent will take an unnecessarily complicated path, while the new agent is able to take the shortest possible path towards the agent. Overall, this new curriculum yields are generally better result than the previous curriculum.

7.5 | Challenges

In this section, we will briefly go through some of the major challenges we faced, and how we can possibly resolve them in the future.

7.5.1 | Training performance does not match actual performance

There are many occasions where the agent seems to be performing reasonably well when one just looking at the training data, but would however, have a very disappointing performance in practice.

By "practice", we don't just mean that the agent does not generalize, we are referring to the fact that even when placed in the same environment they were trained in they *still* perform much more poorly.

We are assuming the root of the problem is that given we are training for $10.0e6$ steps, it is not unusual that a model that perform reasonably well in some niche scenario, "lucked out" by encountering multiple randomly generated environment that just so it happens massively favours the model, and replace models that can perform in a reasonable manner in all scenarios.

For an extended period of time this hindered our development as, at first I was tricked into a belief that there is bug in my code that is leading to this bizarre behaviour. However is luckily resolved in later stage of development where I can finally have confident in my code as I can reliably train a model that does match the performance in training by modifying my training curriculum as mentioned in section 7.4.1. However, before that I have been trying to find a bug that simply does not exists – weeks has been wasted in this to say the least.

As a reflection, I should have added unit tests to my code so I could have confidence that my code is not the reason to this behaviour.

I should also definitely, have had my logging system ready before I started training. With the logging system in place, I could possibly have sufficient information such that I can realize the problem lies within the training process rather than the code I have written for the environment.

7.5.2 | Time Consuming Nature of Reinforcement Learning

RL is a naturally time consuming process, although we are employing many technique to help guide the agent to converge, in a nutshell, RL is still *almost* stochastically searching the optimal parameter for policy. This computational cost is an issue with all machine learning algorithm, but gets even worse in RL as not only do we need to train the agent, but the agent is also required to interact with the environment, implying we need to run an extra process for the environment while we are training.

This combined with the lack of computation resources available to me makes testing painfully slow. The fastest machine available to training will take an average of 14 hours even using grey scale image as input, we also need to keep in mind that we are only training for 10.0×10^6 steps, the box surfing behaviour achieved by Open-AI(Baker et al., 2019) takes 48.1×10^7 steps. Which would approximately take roughly 28 days(about 50 times slower) for us to train. This is less of a problem for OpenAI researchers as they have more computational resources available.

It is not only time consuming to test, it is also time consuming to debug our training process – it is because of the fact that to the most part, even with many effort to interpret why and how a neural network converges(Olden and Jackson, 2002), neural network

and even machine learning in general are significantly harder to debug comparing with more traditional, programmatic approach.

I highly under-estimated the impact of lack of computation power, and I believe that I should have optimized the training process such that it is less computation intensive before I started training, which would significantly mitigate our problem. Another way to mitigate this problem would be to use cloud computing, however this would also bring financial cost to us which is unfortunately not an option to the writer.

7.6 | Conclusions

To conclude, our agent is far less capable than what I would like it to be – one obvious caveats is that throughout the entire process we were never considering the turrets in the training process, thus the agent will not be able to take a path that can safely circumvent all the turrets. Besides, even with this major caveats, the agent is not able to reliably reaching the target at all times. However, the agent is reliable enough that we can use it in the TD game we developed, which means that despite the agents' underwhelming performance, we are still able to have a feature complete game.

I also demonstrated some of the challenges I faced during the development, which implies that if we were able to solve those challenges, it is likely that we can build a RL agent that can be used in commercial game development. Although I was not able to prove that it is practical to use RL in commercial game development, I believe that this attempt shows that there is at least possibility to use RL and gain commercial success.

Results & Discussion & Future Work

8.1 | Overview

In this chapter, we will first highlight our results of this project, and then we will discuss the possible improvement on the performance problem mentioned in section 5.3.2 and the underwhelming performance of our RL agent mentioned in chapter 7.

8.2 | Results

To conclude, we have created a feature complete TD game prototype where its AI is trained with RL. Although the trained RL agent's performance is underwhelming, it is at least in part due to the lack of computation power, and I expect the performance of the agent to significantly improve given enough time and computational resources. Our development also discovered that the significant impact on agent's performance and robustness of training process by changing the input format and the training curriculum.

However, it is indeed the case that this project alone did not prove that RL can be practically used in a commercial game development environment. Yet the fact that we are able to incorporate some of the more advanced technique such as IK without much difficulties, proves that at the very least RL is compatible with game development, and has potential waiting to be explored.

8.3 | Future work – FABRIK

As mentioned in section 5.3.2, due to the inefficiency of the IK solver I implemented with gradient descent, whenever there are 5 or more enemies in the game, the frame rate

significantly dropped to between 10-15 FPS in a machine with a i5-9600k CPU, which is already fairly potent. This is unacceptable given 5 enemies is far from a lot in a TD game – most of the time you can see at least a dozens of them at the same wave.

This can potentially be solved if I switch to **FABRIK**, a heuristic solution towards **IK**. **FABRIK** is often used in game development these years and is known for its efficiency, to put it into context, it is five times faster than the second fastest algorithm Cyclic coordinate descent (**CCD**) known to the community(**Aristidou and Lasenby, 2011**).

I have tried to implement **FABRIK**, however due to time limitations there are bugs within my implementation and I cannot rectify it within the limited time frame. The artefacts of my implementation can still be seen in my project and I am confident that with **FABRIK** in place the performance will significantly improve.

8.4 | Future work – Improvement on Agents

I discussed what technique we used to improve our agent performance in chapter 7. During development of the **RL** agent I attempted to use many other techniques to improve the performance but did not yield good results. Several of these techniques has proven to be useful in various other **RL** tasks and seems to be very promising, however I am forced to stop further investigation due to time limitation. In particular, in the following sections I will discuss 2 of the most interesting ones in details – Hierarchical Reinforcement Learning (**HRL**) which is a fundamentally different approach to **RL**, and **GAIL** which is a type of imitation learning that proves to be potent by other researchers(**Ho and Ermon, 2016**).

8.4.1 | Generative Adversarial Imitation Learning

GAIL is a type of imitation learning inspired by Generative adversarial network (**GAN**). In **GAIL**, instead of only training one agent that makes decision, **GAIL** also trains another model, known as the *discriminator*. The output of discriminator indicates how similar the behaviour of agent is to the experts demonstration. Due to limitation on word count, I have provided a summary over imitation learning in appendix 8.4.2.2, and will refrain our following discussion to **GAIL** specifically.

In **GAIL**, after the agent performs an action, it will receive a reward from the discriminator depending on the similarity between the action taken by agent and the action taken by experts – the more similar it is, the higher the reward. Thus the agent will try to mimic what the expert does in order to maximize potential reward.

However, the longer the discriminator is trained, the better the discriminator gets at distinguishing actions taken by agent from actions taken by experts. Intuitively, it becomes harder and harder for the agent to "fool" the discriminator, and hence the agent will also need to get better at mimicking the action of experts. The competition between the discriminator and the agent leads to the name of "adversarial" imitation learning.

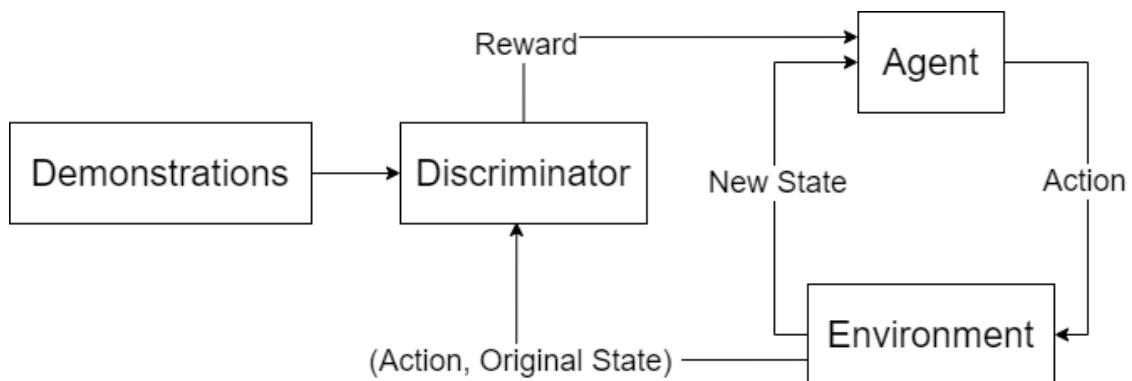


Figure 8.1: Demonstration on how GAIL works

GAIL aims to solve the problem in Behavioral Cloning (BC) where a large amount of demonstrations is required and the risk of compounding error leading to undesired behaviour, whilst avoiding the huge computational cost that roots from the principle of Inverse Reinforcement Learning (IRL)(Ho and Ermon, 2016).

One may note that it is very similar to our original RL framework – the agent take as input a state, output an action and is rewarded for that decision, with the ultimate goal of agent learning a policy that maximizes the reward.

Naturally, one can combine GAIL with other source of reward, in our case, our original extrinsic reward function. After combining with the original extrinsic reward function, the original model from figure 8.1 evolves to figure 8.2.

The ratio between the GAIL reward and the extrinsic reward can be tuned by a hyper parameter. The more we value the GAIL reward, the more important for the agent to mimic the behaviour of the expert, and the quicker in theory it should converge. However it also means that it is less likely to emerge new behaviour which we mentioned in section 6.4.3, which is one of the advantage of using RL to develop the AI.

8.4.1.1 | Results

The result is very surprising, not only the behaviour is almost exactly the same when comparing to not using GAIL, but also the training process itself is almost identical as

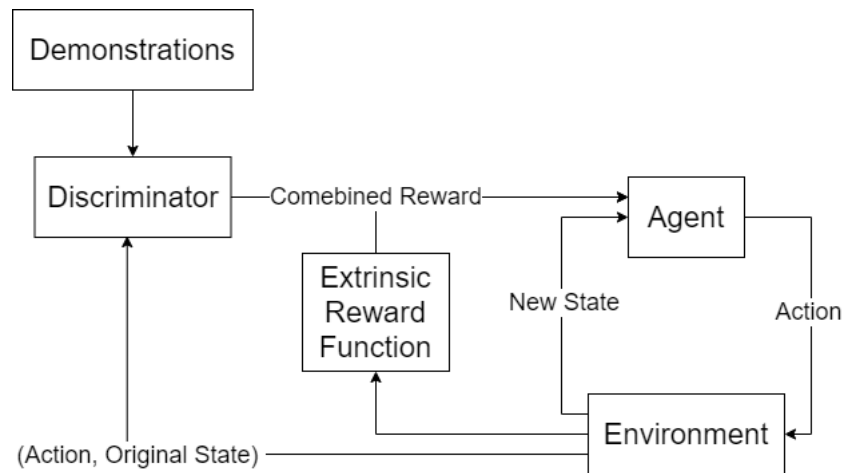


Figure 8.2: Demonstration on how **GAIL** reward can be combined with extrinsic reward function

well. One can even arguably say not using **GAIL** yields a slightly more stable training process judging from figure 8.3.

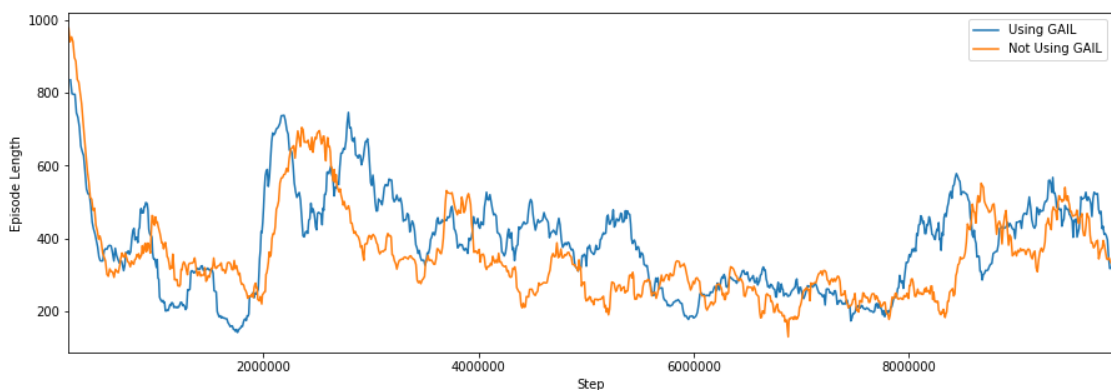


Figure 8.3: Performance Comparison – using **GAIL** vs not using **GAIL**

This might be a result of the fact that I do not have enough demonstration data, such that the agent was not able to deduce the policy I am using when performing the demonstration. Nevertheless I did not have the time to further increase the amount of demonstration training data, and using **GAIL** itself increases the training time required, thus I did not progress with **GAIL**. However further investigating **GAIL** is likely to further improve performance.

8.4.2 | Hierarchy based approach

In section 7.4.1.1, we have discussed how we made our hypothesis of the navigation task being atomic and non-dividable. However as we have already pointed out, due to lack of computation power our testing is not as vigorous as it should be. Hence I was also considering whether the odds that navigation task itself is atomic, but we did not use the correct approach to divide the task in the first place.

After some researching we discovered a new technique, known as [HRL](#) that promotes divide and conquer¹ – in [HRL](#) framework deviates from traditional [RL](#) which only trains one agent for the desired task, divide the task into multiple smaller sub-tasks, and will train an additional agent for each of these sub-tasks. ([Nachum et al., 2018](#)).

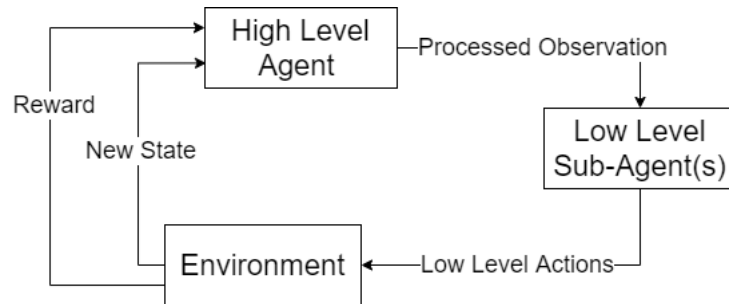


Figure 8.4: A brief diagram describing how [HRL](#) works.

In [Nachum et al. \(2018\)](#)'s research, they demonstrated the possibility to divide the navigation task into two – a task of walking in a straight line, and another task of finding the next desired destination for the agent. An individual agent is trained for each of these tasks. Their testing demonstrated remarkable results and out-performed many other [RL](#) techniques.

Given the high similarity between their test and the nature of our game, their experience show us that it is possible that the task is actually dividable, yet we failed to divide it effectively.

One quite natural advantage of this approach is that it is possible to switch any of the sub-agent to use approaches mentioned in appendix 8.4.2.2. This can be very useful if any of the sub-tasks is easily programmable, we can then focus on training one agent at a time, making training significantly easier.

¹Note OpenAI also suggested another hierarchies based approach in 2017([Frans et al., 2017](#)), this is not to be mistaken as although they both have a similar name, they have a few fundamental differences. Nevertheless their approach is also something that is worth trying.

8.4.2.1 | Results

We decided to carry out a quick test to determine whether HRL is we should prioritize switching into, or are we better off sticking with our original traditional RL.

In our test, we mimic what Nachum et al. (2018) did in their testing – we divide our navigation task into two sub-tasks, with the first task being a high-level "route planner", that is responsible to process the environment and to output a desired state to our low-level "navigator", which is an agent that can *only* walk in a straight line. We hope that by combining both agent we can get an agent that is able to navigate in complicated terrain.

To speed up training, we decided to use a programmatic agent that is guaranteed to put the agent into the desired position to serve as the low-level navigator, and hence only train the route planner agent first, Figure 8.5 briefly demonstrates how are we training our agent. Note this is a fair substitution as we already proofed that we can effectively train agent to walk in a straight line in section 7.12.

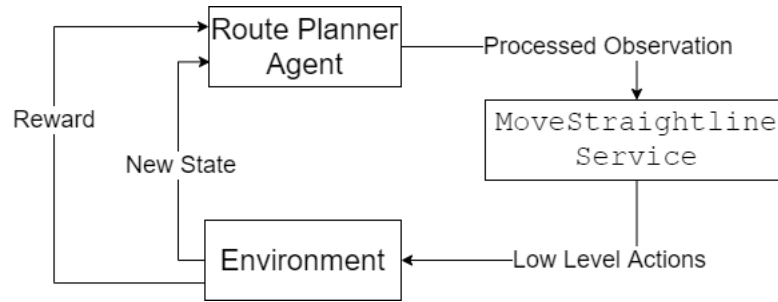


Figure 8.5: A brief diagram describing how we train our route planner agent.

The result is somewhat satisfying, in our case, training has gone fairly smooth looking at the metrics as shown in figure 8.6. The reward steadily increase during training process². In practice, although it still struggles to reach the target in the most optimal manner, it managed to arrive to the target destination eventually when there is no obstacles.

²The drastic drop in around 1.0×10^6 steps is due to the agent progressing to a new level, and is expected as the training get significantly harder. Also note that it is not fair to directly compare the rewards HRL get to the rewards we get previously, as we are using a very different training scheme and the reward configuration is fundamentally different.



Figure 8.6: Cumulative Reward on the planner agent steadily increase during the training process

8.4.2.2 | Caveats

The main reason that we did not pick it over our original training method is that it introduces an additional layer of complexity to our training, and although have an "acceptable" performance when there is no obstacles, it did not surpass what we achieved in section 7.4.1.

Given obstacle avoidance is the key to our project, continuing with [HRL](#) seems unwise. Besides training using [HRL](#) also takes significantly longer – It takes roughly one day to train the route-planner with 5.0×10^6 steps, while it only takes 14 hours to train in traditional [PPO](#) with 10.0×10^6 steps, with significantly better performance. Hence we did not continue investigation with this approach.

References

- Unity ml best practice. <https://github.com/Unity-Technologies/ml-agents/blob/0.11.0/docs/Learning-Environment-Best-Practices.md>, 10 2019. Accessed: 2020-04-23.
- Joachim Ante. Unity at gdc keynote - the future of unity (entity component system & performance) w. joachim ante. GDC, 2018. URL https://www.youtube.com/watch?v=3Mq9EH8RT_U.
- Andreas Aristidou and Joan Lasenby. Fabrik: A fast, iterative solver for the inverse kinematics problem. *Graphical Models*, 73(5):243 – 260, 2011. ISSN 1524-0703. doi: <https://doi.org/10.1016/j.gmod.2011.05.003>.
- Saurabh Arora and Prashant Doshi. A survey of inverse reinforcement learning: Challenges, methods and progress. *CoRR*, abs/1806.06877, 2018.
- Ramon Axelrod. *Navigation Graph Generation in Highly Dynamic Worlds*, chapter 2.12. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.
- Paolo Baerlocher. Inverse kinematics techniques of the interactive posture control of articulated figures. page 156, 2001. doi: 10.5075/epfl-thesis-2383.
- Michael Bain and Claude Sammut. A framework for behavioural cloning. In *Machine Intelligence 15, Intelligent Agents [St. Catherine's College, Oxford, July 1995]*, page 103–129, GBR, 1999. Oxford University. ISBN 0198538677.
- Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. *arXiv*, 2019.
- Raymundo Barrera, Aung Sithu Kyaw, Clifford Peters, and Thet Naing Swe. *Unity AI Game Programming - Second Edition*, chapter 2. Packt Publishing Limited, 2015.
- Steve Butts. Civilization v preview. <https://uk.ign.com/articles/2010/03/08/civilization-v-preview-2>, 2010. Accessed: 2019-10-30.
- David Hernández Cerpa. *The Engagement Decision*, chapter 4.2. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.

- Jack ClarkDario and Amodai. Faulty reward functions in the wild. <https://openai.com/blog/faulty-reward-functions/>, 2016. Accessed: 2019-10-30.
- Mike Geig Cristiano Ferreira. Get started with the unity* entity component system (ecs), c# job system, and burst compiler. Intel’s Documentation, 2018. URL <https://software.intel.com/en-us/articles/get-started-with-the-unity-entity-component-system-ecs-c-sharp-job-system-and-burst-compiler>.
- Xiao Cui and Hao Shi. A*-based pathfinding in modern computer games. *IJCSNS International Journal of Computer Science and Network Security*, 2011.
- Ralph Johnson John Vlissides Erich Gamma, Richard Helm. *Design Patterns: Elements of Reusable Object-Oriented Software*, chapter 1. Addison-Wesley, 1994.
- Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: scalable distributed deep-rl with importance weighted actor-learner architectures. *CoRR*, abs/1802.01561, 2018.
- Richard Fine. Overthrowing the monobehaviour tyranny in a glorious scriptable object revolution. Unite, 2016. URL <https://www.youtube.com/watch?v=6vmRwLYWNRo>.
- Kevin Frans, Jonathan Ho, Xi Chen, Pieter Abbeel, and John Schulman. Meta learning shared hierarchies. *CoRR*, abs/1710.09767, 2017.
- Stephen Grand and Dave Cliff. Creatures: Entertainment software agents with artificial life. *Autonomous Agents and Multi-Agent Systems*, 1998.
- Dale Green. *Procedural Content Generation for C++ Game Development*. 01 2016.
- Jayesh K. Gupta, Maxim Egorov, and Mykel Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In Gita Sukthankar and Juan A. Rodriguez-Aguilar, editors, *Autonomous Agents and Multiagent Systems*, pages 66–83, Cham, 2017. Springer International Publishing. ISBN 978-3-319-71682-4.
- David Hamm. *Navigation Mesh Generation: An Empirical Approach*, chapter 2.12. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.
- Niklas Hansson. Game school gems -influence maps i. <http://gameschoolgems.blogspot.com/2009/12/influence-maps-i.html>, 2009. Accessed: 2019-10-30.
- Ryan Hipple. Overthrowing the monobehaviour tyranny in a glorious scriptable object revolution. Unite Austin, 2017. URL https://www.youtube.com/watch?v=raQ3iHhE_Kk&t=2672s.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning, 2016.
- Nihar R. Mahapatra and Balakrishna Venkatrao. The processor-memory bottleneck: Problems and solutions. *XRDS*, 5(3es):2–es, April 1999. ISSN 1528-4972. doi: 10.1145/357783.331677.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei Rusu, Joel Veness, Marc Bellemare, Alex Graves, Martin Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–33, 02 2015. doi: 10.1038/nature14236.

- Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. *CoRR*, abs/1805.08296, 2018.
- Julian Olden and Donald Jackson. Illuminating the "black box": A randomization approach for understanding variable contributions in artificial neural networks. *Ecological Modelling*, 154:135–150, 08 2002. doi: 10.1016/S0304-3800(02)00064-9.
- Amit Patel. Amit's thoughts on pathfinding - map representation. <http://theory.stanford.edu/~amitp/GameProgramming/MapRepresentations.html>, 2009. Accessed: 2019-10-30.
- Steve Rabin. *AI Game Programming Wisdom 4*, chapter 2. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.
- Steven Rabin. *Game AI Pro 2: Collected Wisdom of Game AI Professionals*, chapter 31.4. A K Peters/CRC Press, 2015.
- Sebastian Ruder. An overview of gradient descent optimization algorithms. *CoRR*, abs/1609.04747, 2016a.
- Sebastian Ruder. An overview of gradient descent optimization algorithms. *CoRR*, abs/1609.04747, 2016b.
- Frantisek Sailer. *Simulation-Based Planning in RTS Games*, chapter 4.5. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.
- Nikolay Savinov, Research Intern, Google Brain Team, Timothy Lillicrap, and Research Scientist in DeepMind. Curiosity and procrastination in reinforcement learning. <https://ai.googleblog.com/2018/10/curiosity-and-procrastination-in.html>, 2018. Accessed: 2019-10-30.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.
- Frederick T. Sheldon, Kshamta Jerath, and Hong Chung. Metrics for maintainability of class inheritance hierarchies. *Journal of Software Maintenance and Evolution: Research and Practice*, 14(3):147–160, 2002. doi: 10.1002/smr.249.
- Nathan Sturtevant. *Memory-Efficient Pathfinding Abstractions*, chapter 2.12. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.
- Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. In *Reinforcement Learning: An Introduction*. The MIT Press, 2014.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2018.
- M.J. Sáiz-Abajo, B.-H. Mevik, V.H. Segtnan, and T. Næs. Ensemble methods and data augmentation by noise addition applied to the analysis of spectroscopic data. *Analytica Chimica Acta*, 533(2):147 – 159, 2005. ISSN 0003-2670. doi: <https://doi.org/10.1016/j.aca.2004.10.086>.
- Marcus Toftedahl and Henrik Engström. A taxonomy of game engines and the tools that drive the industry. 08 2019.
- David A van Dyk and Xiao-Li Meng. The art of data augmentation. *Journal of Computational and Graphical Statistics*, 10(1):1–50, 2001. doi: 10.1198/10618600152418584.

- Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg¹, Alexander S. Vezhnevets¹, Rémi Leblond¹, Tobias Pohlen¹, Valentin Dalibard¹, David Budden¹, Yury Sulsky¹, James Molloy¹, Tom L. Paine¹, Caglar Gulcehre¹, Ziyu Wang¹, Tobias Pfaff¹, Yuhuai Wu¹, Roman Ring¹, Dani Yogatama¹, Dario Wünsch², Katrina McKinney¹, Oliver Smith¹, Tom Schaul¹, Timothy Lillicrap¹, Koray Kavukcuoglu¹, Demis Hassabis¹, Chris Apps, and David Silver. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 2019.
- Baylor Wetzal. *The Engagement Decision*, chapter 4.8. Course Technology PTR, 25 Thomson Place, Boston, MA 02210, USA, 2014.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(3–4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/BF00992696.

Common Implementation on Game AI

In this appendix, I will briefly describe some of the existing approach to implement a tactical AI in a RTS game. These approaches is not directly used during our development, but demonstrated some of the common caveats in the current implementation of game AI

Influence Map

Influence maps are a technique developed back in the 1990s and is often adapted as a tile-like abstraction. It is a powerful tool that allows programmers to encode historical statistics, future predictions and ultimately draw a summary from existing data such as ally density and unit's individual strength, where the AI will be able to use it in decision-making process. A simple influence map for a game of Go can be seen in figure 1.

Through influence map, it is easy to implement a fuzzy logic like behaviour - this is particularly useful in RTS games where units can perform "soft-retreating" (backing away to increase distance between the unit and its source of threat) to cover and evade danger or to rout from battlefield to regroup and fight another day.

For the above reasons, it is very commonly used to create an illusion of intelligent, which in most case is all we need - after all as long as the player think the AI is intelligent, it does not really matter if it really is.

However, it is worth noting that influence maps are merely a data type - they only helps to encode data of the battlefield into a form that can be easily interpreted by AI, but without some other tools to utilize this data it will be nothing more than a set of data. In practice, it is often embedded into the navmesh by increasing the weight of edges to

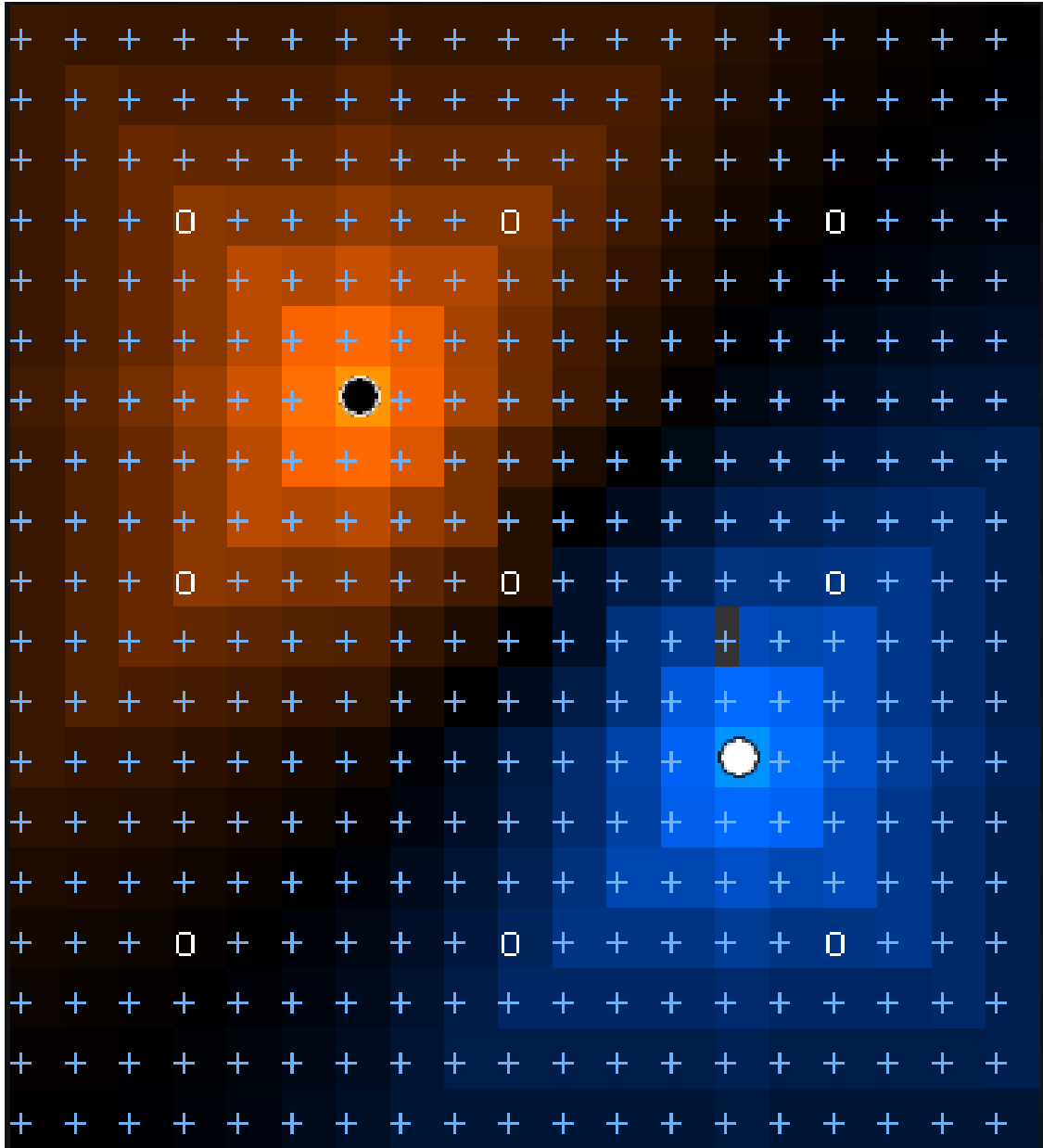


Figure 1: Influence map of Go created by [Hansson \(2009\)](#), blue represents influence of white pieces, orange represents influence of black pieces

nodes that has high influence of the opponent, and hence make the path less appealing to the pathfinding algorithms.

However, performance problem can occur when the navmesh itself is highly dynamic or when the influence map itself need to be frequently updated as the units move around. It is because if either of the cases is true, the AI will need to frequently update its current path to the destination in order to navigate to the destination as intelligently as possible. This problem could be solved by reducing the frequency of updating the influence map or the frequency of the AI checking for the best possible path.

Another problem is that it is often argued that influence map is a form of cheating AI - it possess knowledge and information that it should not have, for example, the presence of the player's units will be available to the AI through their influence on the map, even if the unit itself is not within the sights of the AI's units.

Whilst an easy countermeasure would be to not encode information that is not available to the AI, this unfortunately leads to stupid AI as unlike humans, machine does not have the wisdom to predict and anticipate. Insufficient information often leads to AI making dumb decisions such as allocating defense or mustering an attack in locations that is just simply non-sensical. The difficulty of balancing is also suggested by Rabin (2015).

The fact that influence maps are merely a data type also makes them implementation-specific, and tuning would be necessary to develop a decent influence map that represents the situation of the field correctly, again, requires a huge amount of effort.

Tactic and Planning

In a simple strategy game, even random tactics employed by AI might be perceived as intentional by human players (Wetzel, 2014). This is due to the fact that human tends to rationalize any observation on their surroundings.

However, random tactics will soon look non-sensible as the game's complexity increases. For this reason, a lot of effort has been invested to help AI make better decisions. It is still somewhat manageable when the game's information is *complete*, such as chess and Tic-Tac-Toe, yet once the information is *incomplete* (i.e, not all information is available to all players), it quickly gets out of control.

Most common implementations nowadays relies on an intensive amount of maths and/or custom tuning by designers. We will discuss some of the implementation below.

Finite-State Machine

As mentioned by Cerpa (2014), Finite-State Machines (FSMs), such as example shown in 2, are widely used for programming behaviour and planning systems. They are easy to implement and can be very effective, especially for less complicated games. Moreover, it forces the programmer to think of every edge case, which although prevents emergent behaviour, reduce bugs in production environments. Besides the AI can be relatively easy to customize as the game develops, from change of general behaviour to a performing specific action.

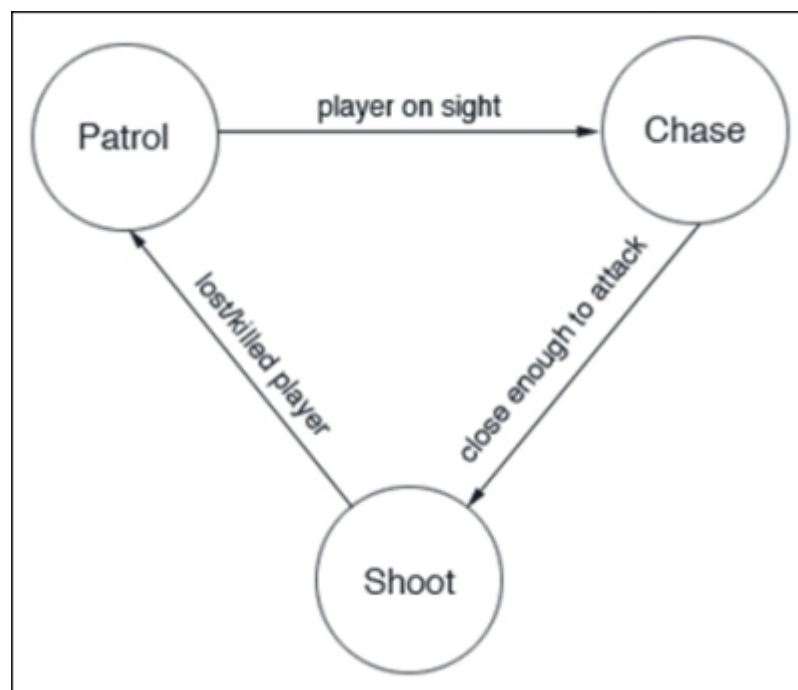


Figure 2: Simple FSM created by Barrera et al. (2015)

However, FSMs do not scale very well and can easily get unmanageable in large and complex projects. Besides as there is no emergent behaviour, the player will find it repetitive and can easily find patterns after multiple playthroughs, thus massively reduces re-playability.

Goal-Oriented Action Planning

Goal Oriented Action Planning (GOAP) is an abstraction that helps AI to plan a series of *actions* to achieve a desired state. Each action can have both *preconditions* and *effects*, by chaining a series of effects to accomplish different preconditions, the agent will

eventually find an action that can be performed and has the desired effects to change the environment such that it satisfies a desired state.

In practice, it is often used for complicated AI and has a decent result. It is also much more scalable as the programmer does not need to manage a huge FSM. It is often used to create AI that would be too complicated for FSM to handle.

However, it again requires a designer or gameplay programmer to create every possible action and specify their relative cost - which again, requires a lot of effort and fine-tuning. Besides, the biggest weakness of GOAP is perhaps *someone* will need to assign a goal for it such that it can search for the best series of action to be taken in order to satisfy the goal, which brings us back to our initial question: how do we find out what is our goal?

To counteract the problem of goal finding, Cerpa (2014) has suggested an approach where goals are driven by motivation - a high-level process that detects needs and creates goals to be accomplished.

A similar approach is also employed in Civilization V (Butts, 2010) where the army's movement is separated into a top-down hierarchies - *Grand Strategic AI* decides how to win a game, *Strategic AI* manage empires and city as a whole, *Operational AI* decides which battle to fight and allocate necessary forces, and *Tactical AI* that controls units in a local scale. In practice, this extra layer often produce good enough results.

Simulation-Based Planning

In contrast to the above more mathematical approach, Sailer (2014) has suggested an empirical, simulation-based approach for RTS. They demonstrated that it can be run efficiently through some optimization, and can adapt to human player's frequently changing tactics.

Simulation-Based Planning functions by simulating possible outcomes after the AI employs a certain tactic, and maximizing the payoffs. It also uses opponent modeling to exploit players' weaknesses. To avoid the search space being enormous, it restricts the AI choice to a set of predefined policies such that the program could be run efficiently in real-time.

Yet once again, it requires the designer or the AI programmer to create every possible policy in advance, requiring massive resources to be invested into fine-tuning. Besides it is not known whether it is suitable for more complicated games - after all Sailer (2014) has only tested this approach on a very simple RTS.

Summary

As we can see, most of the current approaches used for RTS's AI are likely to requiring a huge amount of time and resources to fine-tune, which, in some cases, might not be possible. It is also fair to say most of them ultimately requires some input from other subsystem that provides it with a certain goal or motivation. Which is, however, one of our questions in the first place.

Imitation Learning

Imitation learning is based on the idea that sometimes it is intrinsically difficult for the agent to discover the optimal policy. So instead of throwing the agent into the environment without any guidance, we provide the agent with some *demonstration* to show how the task should be done.

There are various ways to achieve imitation learning, we will briefly discuss two of which that is widely used, and discuss [GAIL](#) which is used in our development in a later section.

Behavioral Cloning

BC is one of the simplest implementation of imitation learning. Proposed by [Bain and Sammut \(1999\)](#), it has been proven that BC can learn tasks very quickly with reasonable performance. BC is a supervised learning technique that heavily depends on human demonstrations. In BC, the agent is trained to learn a policy that mimics the policy demonstrated by the *expert*(often a human), which is particularly useful for tasks which are hard to capture by a reward function but can be easily demonstrated. Some typical example including flying a helicopter or driving a car.

However, the downside of BC is that if the agent is put into a state that is never encountered by the experts, there is no basis for the agent to learn or reference from. This can happen when small errors adds up and leading the agent to a state that the expert has never visited before([Ho and Ermon, 2016](#)). As a result BC often needs a large amount of data for it to learn effectively, which can be hard and costly to come by in some situation. One way to make up for the enormous amount of data required is to use data augmentation ([van Dyk and Meng, 2001](#)), which is a widely used technique in machine learning to help get enough data for the algorithm to function properly.

Common augmentation technique for images augmentation includes adding noise to the images(Sáiz-Abajo et al., 2005).

Inverse Reinforcement Learning

IRL is fundamentally different from BC - instead of learning the policy from the expert, IRL attempts to learn the reward function behinds the expert's policy.

The reasoning behind this is that it can be difficult to model the reward function that guide to a correct behaviour(Arora and Doshi, 2018), so instead of modeling the reward function manually, we first learn the reward function of the expert - or in other words, we learn the *intent* of the expert. Only after that do we continue with our usual training. Note that after IRL it does not matter what algorithm we used to optimize the policy. The fact that we did not define the reward function, but rather learn the reward function from the experts, gain the name of *inversed RL*.

The idea of IRL is elegant, yet the problem lies with the immense computational cost as IRL essentially requires solving a forward problem(finding the optimal policy) in an inner loop of finding the optimal cost(reward) function(Ho and Ermon, 2016). Besides, we should also note that IRL does not directly yield the policy, and learning the policy even when given the correct reward function may not be trivial.

Alternatives on Software Architecture

In the following sections, we will discuss some of the valid alternatives for our decision, and compare them with our chosen options.

Unity Events

There are many different event management system, a popular alternative with Unity is to use `UnityEvent`.

Unity Events has the advantage that it is GUI-friendly - one can configure the events through dragging and dropping in Unity's inspector as shown in 3.

In contrast, event aggregator provided by [CM](#) requires the subscriber to implement a specific interface `IHandle<T>` in which `T` is the type of subscribed event, and until the object unsubscribed itself, it will receive a callback from the event aggregator everytime event of type `T` is published. It is worth mentioning that [CM's](#) event aggregator is reflection-based³, which means that it is slightly more computationally expensive.

It is apparent that [CM](#) and Unity takes a very different approach to the problem, [CM](#) has decided the configuration *must* stay with code, while Unity allows configuration through Graphical User Interface (GUI). However they still follow the same principle - "subscribers" subscribe itself to a certain type of event, and receives a notification whenever it is published by any publisher.

`UnityEvent`'s GUI-support do come with a cost. Such as:

1. Requiring the subscriber to be serializable by Unity (we will elaborate more about this later in section 4.2)

³"Reflection" refers to the ability of a class to examine and modify its own structure. Please refer to the [official documentation](#) for more details.

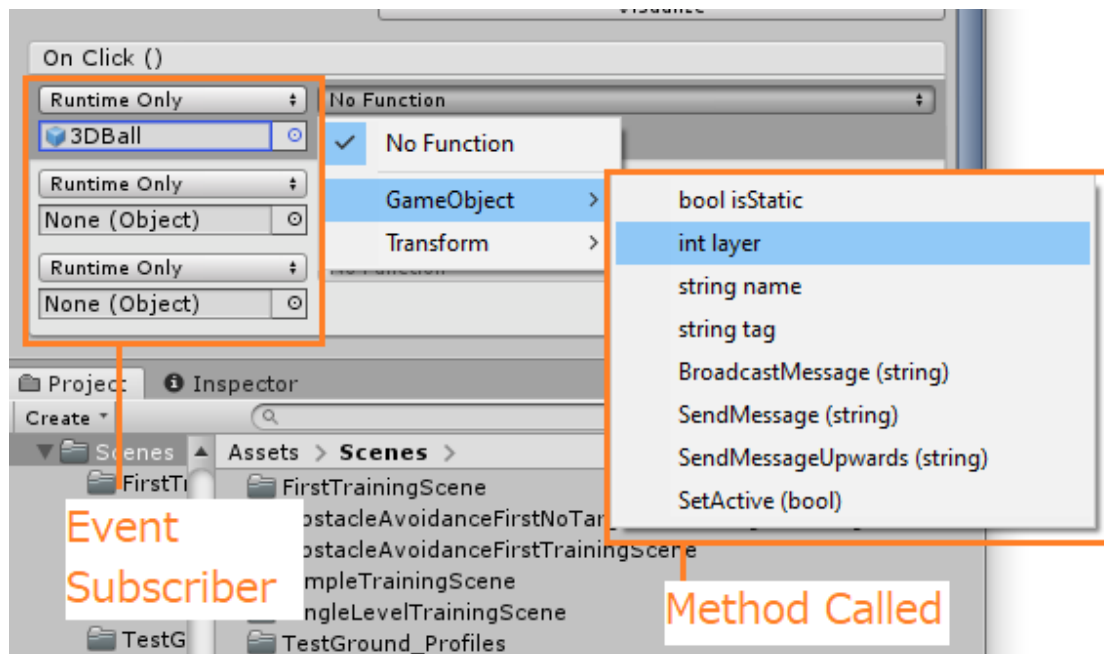


Figure 3: Demonstration of how UnityEvent is used

2. The called method must be public, which weakened the promise of having no direct dependency
3. Suffering from the same limitation to standard *delegates*⁴, for example it can prevent garbage collection as it holds a reference to the subscriber, potentially leading to memory leak.

In contrast, CM's event aggregator will elegantly remove any "dead" reference to prevent memory leaking. Besides, it also supports inheritance in the subscribed events, making CM's event aggregator significantly more flexible.

It is also arguable if being GUI-friendly is *truly* an advantage in our scenario, as configuring through inspector can be very time consuming especially if you need to configure every single component. This is not to say GUI does not bring its own advantage, but we have to keep in mind that Unity was built for different types of professions - from designer, artist to programmer, having all events configurable through GUI obviously makes life a lot easier for designer and artists. However it is not necessarily helpful in our case when we only have one programmer.

⁴Please refer to the [official documentation](#) for more information on delegate

Entity Component System

ECS is the other very potent alternative over **CBA**. It is important to note that although they share the same terminology, "components" in **ECS** behaves in a complete different manner.

In **ECS**, similar to **CBA**, every object in the game is known as an Entity, which is essentially an object that simply contains different Components, which in contrast with **CBA**, does not encapsulate the code required to implement the component's behaviour, but rather contains raw data from that is required for the calculation that is done by the System, that runs in every frame and does calculation based on every existing components' data. Figure 4 demonstrates how movement can be handled in **ECS**.

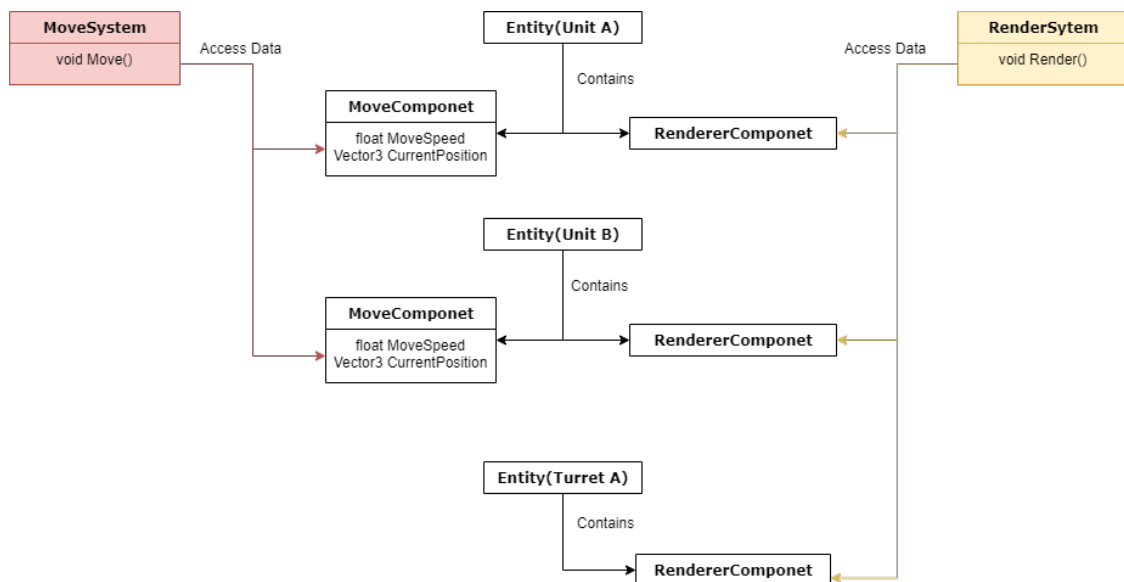


Figure 4: Demonstration of how **ECS** can handle our movement code, RendererComponent's data is not included for simplicity

ECS shares the benefits from **CBA**, it is flexible and easily maintainable. Moreover, **ECS** in nature helps writing code in a cache-friendly data layout, as in **ECS**, all components of same type are stored in a contiguous array, which helps to maximize benefits from prefetching and minimize cache misses(which is computationally expensive) - this has become the key for performance as memory's improvement has been slower relative to the improvement in processors(Mahapatra and Venkatrao, 1999). This approach promotes efficient usage of caches and is one implementation of *data-oriented design*. In contrast **CBA** will often have a more "scattered" memory layout as shown in 5.

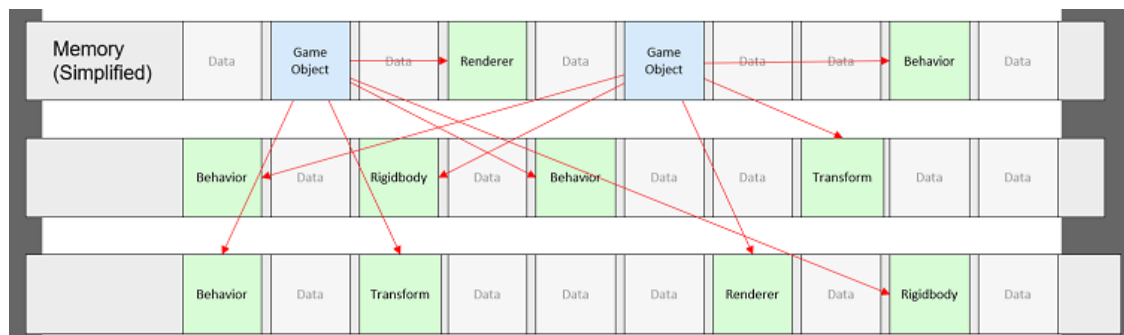


Figure 5: Typical memory layout in CBA (Cristiano Ferreira, 2018)

Consider a concrete case where in our game we needed to move our units, in CBA we will need access to the Transform component - an Unity built in component that contains information regarding an objects position, orientation and scales, which *also* contains many other data such as its parent transform and the `GameObject` that contains it. One can immediately sees that many of that data is not required for a moving operation. In contrast, using ECS we will only access the necessary data, and all of those data will be contagious to each other as they are accessed in a chunk. Figure 6 shows a simplified memory layout in both situation.

Furthermore, Unity also decided to add Job system, which helps abstracting the complicated nature of multi-threaded programming, and a burst compiler which optimizes machine code around the aforementioned technology. The combined ecosystem is known as Data-oriented technology stack (DOTS). Ante (2018), Chief Technology Officer from Unity, claimed that it can increase speed by 100 times. Benchmark from Intel (Cristiano Ferreira, 2018) also shows drastic improvement in their simple testing environment - such that the frame rate became GPU bound which is rare for such a simple environment. Although it is worth noting that both results are conducted by related parties, and thus should be taken with a grain of salt, there is still no doubt that the improvement is significant.

ECS and the DOTS essentially has all the advantage of CBA, plus being significantly faster, although with the slight drawbacks of being slightly harder to reason with. The only reason I did not use it over CBA is that it is still under preview.

Even with community consensus that it is in a rather stable state, and there are studios⁵ in the industry using ECS at the moment. I decided it is too much of a risk

⁵Tic Toc Games, Far North Entertainment, Nordeus, Freejam, and Moonlight Mouse are using DOTS at the time of writing, as shown in Unity's official website

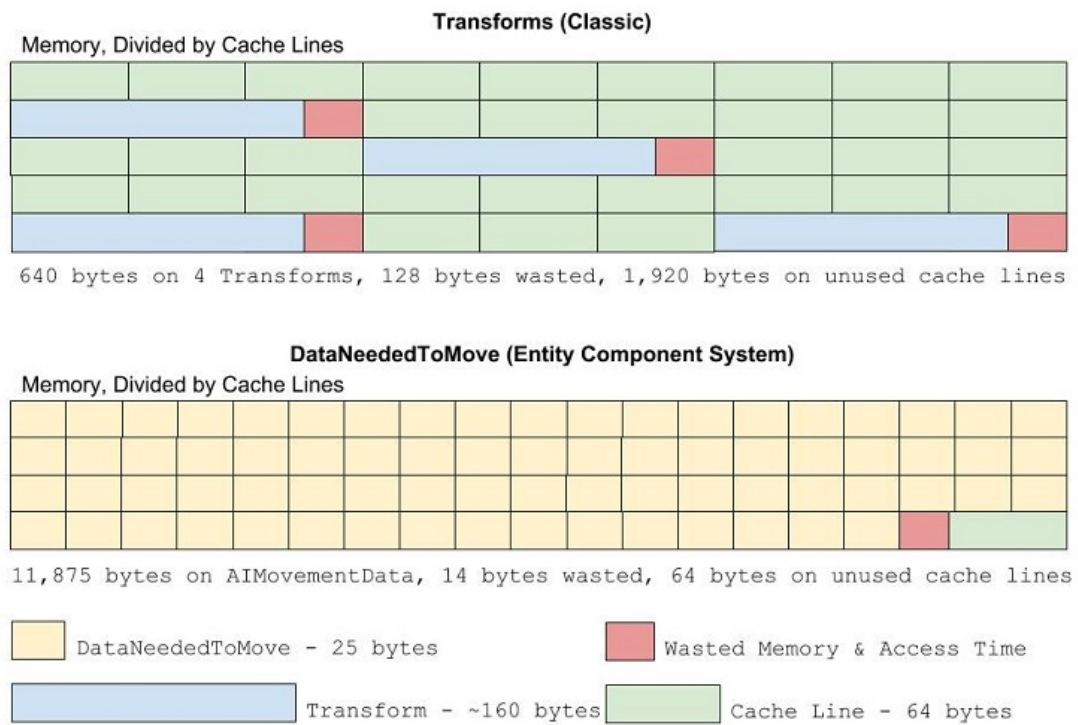


Figure 6: Comparison of memory layout between ECS and CBA when moving a GameObject (Cristiano Ferreira, 2018)

combining with the fact that Unity ML-Agents that is our key library for this project, is also under preview. Running two package under active development can easily leads to disastrous results, for example, Unity ML-Agents has released ten different versions in the course of six months, in which three of them is introducing breaking changes.

Given how rapidly Unity ML-Agents is changing, I have to make the difficult decision on not using DOTS and ECS, however they are definitely worth exploring, and will be one of the possible improvements for this project.